

UPS 2025 Annual Shipping Performance

Building the Dashboard by Hand

Complete Power Query and DAX, start to finish, in Power BI Desktop

What this builds: 8 tables · 11 relationships · 112 measures, reading your four UPS Excel extracts directly.

What you need: Power BI Desktop and your four .xlsx files. Nothing else — no template, no Python, no CSV files, no downloads.

How long: about an hour for the model, plus report layout.

Everything here is also on GitHub

If you can open a browser, you do not have to retype anything from this document. The same code is at github.com/CrawfordTS25/CrawfordTS25, branch **claude/executive-dashboard-build-09ai41**, under `ups-2025-executive-dashboard/`. Open a file, click *Raw*, select all, copy. That is faster and safer than copying out of a PDF.

Why build by hand rather than open the template

The .pbix template does the same job in five minutes, and it is the better route if you can get the file onto the machine. This document exists for when you cannot. It is not a lesser path: the model it produces is identical, and it reads your raw Excel files directly rather than going through intermediate CSVs, so there is one less moving part.

1. Before you start

1.1 Put the four files in one folder

The queries reference these exact names. Rename your extracts and put all four in a single folder:

Rename to	This is your...
UPS_2025_01_VolumeSpend.xlsx	January file
UPS_2025_02_VolumeSpend_YTD.xlsx	February file — the year-to-date one
UPS_2025_03_AllTabs.xlsx	March file — the one with TnT / Accessorial / Claims / VnS tabs
UPS_2025_04_VolumeSpend.xlsx	April file

Note the folder path. You will need it in the next step, and it must be absolute — for example C:\UPS\2025, with no trailing backslash.

1.2 How the pieces fit together

Nine queries, built in order. Each one that references another must come after it.

#	Query	Reads	Depends on
1	p_Folder	— (parameter)	—
2	Dim_ServiceMapping	nothing — inline table	—
3	Fact_VolumeSpend	all four files	1, 2
4	Fact_TimeInTransit	March file, TnT tab	1, 2
5	Fact_Claims	March file, Claims tab	1
6	Fact_Accessorial	March file, Accessorial tab	1, 2
7	Dim_Date	nothing — generated	—
8	Dim_Account	the fact queries	3, 4, 5
9	Dim_Service	the mapping table	2

1.3 The pattern for every query

1. In Power BI Desktop: **Home > Transform data** to open Power Query.
2. **Home > New Source > Blank Query**.
3. Rename it in the Query Settings pane on the right — the name must match *exactly*, because later queries reference it.
4. **Home > Advanced Editor**.
5. Delete everything in the box, paste the query, click **Done**.

Paste the whole block including the comment banner — Power Query keeps comments and they are the record of why each rule exists.

If a query shows an error

Do not delete it and move on. Every query here depends on the ones before it, so one broken query cascades. The error message names the step that failed; check that step's line against this document. Section 8 lists the errors you are most likely to hit and what each one means.

2. Create the parameter

This is a **parameter**, not a query. In Power Query: **Home > Manage Parameters > New Parameter**.

Field	Value
Name	p_Folder
Type	Text
Current Value	the folder holding your four files, e.g. C:\UPS\2025

No trailing backslash

The queries build paths as `p_Folder & "\" & fileName`. A trailing backslash produces a doubled separator and a file-not-found error naming a path with `\\` in the middle — which at least tells you exactly what went wrong.

3. The queries

Nine blocks. Create each as a Blank Query, name it exactly as shown in its banner, and paste the code into Advanced Editor.

Query 2 — Dim_ServiceMapping

```
// =====
// 1. PARAMETER p_Folder
// =====
// This is a PARAMETER, not a query. Create it via:
// Home > Transform data > Manage Parameters > New Parameter
// Name: p_Folder
// Type: Text
// Current Value: the folder holding your four .xlsx files
// e.g. C:\UPS\2025
// No trailing backslash.

// =====
// 2. QUERY Dim_ServiceMapping
// =====
// Home > New Source > Blank Query, rename to Dim_ServiceMapping, then
// Home > Advanced Editor and paste everything below this banner.
//
// This is the two-vocabulary bridge and it is the reason the dashboard can put
// cost and reliability for the same service on one row.
//
// The Volume & Net Spend extract calls a service "NDA REG / EXPRESS PKG".
// The Time-in-Transit extract calls the same service "NEXT DAY AIR". Neither
// list contains the other's terms, so a single product column cannot join them.
// Keying on (SourceSystem, RawProduct) lets both vocabularies live in one table
// and resolve to one ServiceGroup.
//
// It is inlined as a literal table rather than read from a CSV so the model has
// no external dependency beyond your four Excel files. To change a mapping,
// edit the row here.
let
    Source = #table(
        type table [
            SourceSystem = text, RawProduct = text, ServiceGroup = text,
            ServiceGroupShort = text, ServiceTier = text, Scope = text,
            IncludeInRanking = text, ServicePriority = Int64.Type
        ],
        {
            {"VnS", "GROUND PKG",
             "Ground", "Ground", "Deferred", "Domestic", "Yes", 1},
            {"VnS", "GROUND CWT",
             "Ground", "Ground", "Deferred", "Domestic", "Yes", 1},
            {"VnS", "STANDARD GROUND",
             "Ground", "Ground", "Deferred", "Domestic", "Yes", 1},
            {"VnS", "3DS PKG",
             "3 Day Select", "3DS", "Deferred", "Domestic", "Yes", 2},
            {"VnS", "2DA REG / EXPEDITED PKG",
             "2nd Day Air / Expedited", "2DA", "Expedited", "Domestic", "Yes", 3},
            {"VnS", "2DA REG / EXPEDITED LTR",
             "2nd Day Air / Expedited", "2DA", "Expedited", "Domestic", "Yes", 3},
            {"VnS", "2DA REG / XPD LTR BILLED AS PKG",
             "2nd Day Air / Expedited", "2DA", "Expedited", "Domestic", "Yes", 3},
            {"VnS", "2DA AM PKG",
             "2nd Day Air Early A.M.", "2DA AM", "Expedited", "Domestic", "Yes", 4},
            {"VnS", "2DA AM LTR",
             "2nd Day Air Early A.M.", "2DA AM", "Expedited", "Domestic", "Yes", 4},
            {"VnS", "2DA AM LTR BILLED AS PKG",
             "2nd Day Air Early A.M.", "2DA AM", "Expedited", "Domestic", "Yes", 4},
            {"VnS", "NDA PM / EXPRESS SAVER PKG",
             "Next Day Air Saver", "NDA Saver", "Premium", "Domestic", "Yes", 5},
            {"VnS", "NDA PM / EXPRESS SAVER LTR",
             "Next Day Air Saver", "NDA Saver", "Premium", "Domestic", "Yes", 5},
            {"VnS", "NDA PM / XPR SVR LTR BLD AS PKG",
             "Next Day Air Saver", "NDA Saver", "Premium", "Domestic", "Yes", 5},
            {"VnS", "NDA REG / EXPRESS PKG",
             "Next Day Air / Express", "NDA", "Premium", "Domestic", "Yes", 6},
            {"VnS", "NDA REG / EXPRESS LTR",
             "Next Day Air / Express", "NDA", "Premium", "Domestic", "Yes", 6},
            {"VnS", "NDA REG / EXPRESS CWT",
             "Next Day Air / Express", "NDA", "Premium", "Domestic", "Yes", 6},
            {"VnS", "NDA REG / XPR LTR BILLED AS PKG",
             "Next Day Air / Express", "NDA", "Premium", "Domestic", "Yes", 6},
            {"VnS", "NDA AM / EXPRESS PLUS PKG",
             "Next Day Air Early", "NDA Early", "Premium", "Domestic", "Yes", 7},
            {"VnS", "NDA AM / EXPRESS PLUS LTR",
             "Next Day Air Early", "NDA Early", "Premium", "Domestic", "Yes", 7},
            {"VnS", "WW EXPRESS REG PKG",
             "International Express", "Intl Express", "Premium", "International", "Yes", 8},
            {"VnS", "WW EXPRESS REG LTR",
             "International Express", "Intl Express", "Premium", "International", "Yes", 8},
            {"VnS", "WW EXPRESS REG DOC",
             "International Express", "Intl Express", "Premium", "International", "Yes", 8},
            {"VnS", "WW EXPRESS REG PAK",
             "International Express", "Intl Express", "Premium", "International", "Yes", 8},
            {"VnS", "WW EXPRESS PM / SAVER PKG",
             "International Express Saver", "Intl Saver", "Premium", "International", "Yes", 9},
            {"VnS", "WW EXPRESS PM / SAVER LTR",
             "International Express Saver", "Intl Saver", "Premium", "International", "Yes", 9},
            {"VnS", "WW EXPRESS PM / SAVER DOC",
             "International Express Saver", "Intl Saver", "Premium", "International", "Yes", 9},
            {"VnS", "WW EXPRESS PM / SVR LTR BLD AS DOC",
             "International Express Saver", "Intl Saver", "Premium", "International", "Yes", 9},
        }
    )
```

```

    {"VnS", "WW EXPEDITED / 2DA PKG",
     "International Expedited", "Intl Expedited", "Expedited", "International", "Yes", 10},
    {"VnS", "WW EXPEDITED / 2DA LTR",
     "International Expedited", "Intl Expedited", "Expedited", "International", "Yes", 10},
    {"VnS", "WW 3DS PKG",
     "International Standard", "Intl Standard", "Deferred", "International", "Yes", 11},
    {"VnS", "MISC / UNC",
     "Excluded - Adjustments", "Excluded", "Non-Service", "Not Applicable", "No", 99},
    {"TnT", "GROUND",
     "Ground", "Ground", "Deferred", "Domestic", "Yes", 1},
    {"TnT", "THREE DAY SELECT",
     "3 Day Select", "3DS", "Deferred", "Domestic", "Yes", 2},
    {"TnT", "SECOND DAY AIR",
     "2nd Day Air / Expedited", "2DA", "Expedited", "Domestic", "Yes", 3},
    {"TnT", "SECOND DAY EARLY AM",
     "2nd Day Air Early A.M.", "2DA AM", "Expedited", "Domestic", "Yes", 4},
    {"TnT", "NEXT DAY AIR SAVER",
     "Next Day Air Saver", "NDA Saver", "Premium", "Domestic", "Yes", 5},
    {"TnT", "NEXT DAY AIR",
     "Next Day Air / Express", "NDA", "Premium", "Domestic", "Yes", 6},
    {"TnT", "UPS EXPRESS EARLY",
     "Next Day Air Early", "NDA Early", "Premium", "Domestic", "Yes", 7},
    {"TnT", "INTERNATIONAL EXPRESS",
     "International Express", "Intl Express", "Premium", "International", "Yes", 8},
    {"TnT", "INTERNATIONAL EXPRESS SAVER",
     "International Express Saver", "Intl Saver", "Premium", "International", "Yes", 9},
    {"TnT", "INTERNATIONAL EXPEDITED",
     "International Expedited", "Intl Expedited", "Expedited", "International", "Yes", 10},
    {"TnT", "INTERNATIONAL STANDARD",
     "International Standard", "Intl Standard", "Deferred", "International", "Yes", 11}
  })
in
  Source

```

Query 3 — Fact_VolumeSpend

```

// =====
// 3. QUERY Fact_VolumeSpend
// =====
// New Source > Blank Query > rename to Fact_VolumeSpend > Advanced Editor >
// paste everything below this banner.
//
// UPS ships two different Volume & Net Spend layouts and this query reads both:
//
// Schema A - January and February files
// Header on row 6, TY/LY/Diff sub-header on row 7, data from row 8.
// Carries a WE/Mo/Qtr period column and prior-year comparatives, so each
// measure appears three times across the row (this year / last year /
// difference). Only the this-year columns are taken.
//
// Schema B - March and April files ("VnS" sheet)
// Header on row 5, data from row 6. One period, no comparatives.
//
// -----
// THE YEAR-TO-DATE TRAP - the most important thing in this query
// -----
// The February file is a YEAR-TO-DATE extract. It contains January AND February.
// The January file contains January.
//
// Loading both wholesale double-counts January exactly: a 100% overstatement of
// that month, roughly 40% of the loaded period, with no error, no warning, and
// a total that still looks entirely plausible.
//
// This query avoids it structurally rather than cleverly: the February step
// filters to month 2 only, and January comes from the January file. Nothing to
// deduplicate, nothing to get wrong.
//
// If a future monthly file also arrives as year-to-date, copy the February step
// and filter it to its own month the same way.
// =====

let
  // --- local helpers -----

  // UPS prefixes ID-like columns with an apostrophe to stop Excel reformatting
  // them. Left in place, "ABC123" and "ABC123" are different keys and the
  // account relationship silently half-fails.
  Clean = (v) as text =>
    let
      t = if v = null then "" else Text.Trim(Text.From(v))
    in
      Text.Trim(if Text.StartsWith(t, "'") then Text.RemoveRange(t, 0, 1) else t),

  Num = (v) as number => if v = null then 0 else (try Number.From(v) otherwise 0),

  // Drop report artefacts only - Grand Total lines and rows with no account.
  // Deliberately NOT keyed on the sub-parent: UPS masks it with "@@" on some
  // accounts, and those rows carry a real account number and real shipment
  // volume. Filtering on "@@" - the obvious move - silently deletes genuine
  // international activity.
  KeepRow = (sub, acct) as logical =>
    let
      s = Text.Lower(Clean(sub)),
      a = Text.Lower(Clean(acct))
    in
      not (s = "grand total" or s = "total" or a = "grand total" or a = "total")
      and not (a = "" and s = ""),

  SubNameOf = (numv, namev) as text =>
    if Clean(numv) = "@@" or Clean(namev) = "@@"
    then "Unassigned Sub-Parent"

```

```

else Clean(namev),

MaskedOf = (numv, namev) as text =>
  if Clean(numv) = "@@" or Clean(namev) = "@" then "Yes" else "No",

FirstSheet = (fileName as text) as table =>
  Table.SelectRows(
    Excel.Workbook(File.Contents(p_Folder & "\" & fileName), null, true),
    each [Kind] = "Sheet"){0}[Data],

// --- Schema A: January / February layout -----
// Column offsets, verified against the real extracts:
// Column2 Sub Number      Column3 Sub Name
// Column4 Account Number  Column5 Account Name
// Column6 City            Column7 State
// Column8 Product         Column9 WE/Mo/Qtr (period)
// Column10 Volume (TY)    Column19 Gross Spend (TY)
// Column22 Net Spend (TY) Column25 Incentive (TY)
// Column31 Billed Weight Lbs (TY)
ReadSchemaA = (fileName as text, monthNo as number) as table =>
  let
    Rows = Table.Skip(FirstSheet(fileName), 7),
    Pick = Table.SelectColumns(Rows, {
      "Column2", "Column3", "Column4", "Column5", "Column6", "Column7",
      "Column8", "Column9", "Column10", "Column19", "Column22",
      "Column25", "Column31" }),
    Named = Table.RenameColumns(Pick, {
      {"Column2", "SubNumberRaw"}, {"Column3", "SubNameRaw"},
      {"Column4", "AccountNumberRaw"}, {"Column5", "AccountNameRaw"},
      {"Column6", "CityRaw"}, {"Column7", "StateRaw"},
      {"Column8", "ProductRaw"}, {"Column9", "PeriodRaw"},
      {"Column10", "VolumeRaw"}, {"Column19", "GrossRaw"},
      {"Column22", "NetRaw"}, {"Column25", "IncentiveRaw"},
      {"Column31", "WeightRaw"} }),
    // The period column is authoritative here. A YTD file holds several
    // months, so this filter is what keeps January out of the February load.
    Kept = Table.SelectRows(Named, each
      KeepRow([SubNumberRaw], [AccountNumberRaw])
      and Clean([ProductRaw]) <> ""
      and Num([PeriodRaw]) = monthNo),
    Shaped = Table.AddColumn(Kept, "Rec", each [
      MonthNumber      = monthNo,
      SubNumber        = Clean([SubNumberRaw]),
      SubName          = SubNameOf([SubNumberRaw], [SubNameRaw]),
      IsMaskedSubParent = MaskedOf([SubNumberRaw], [SubNameRaw]),
      AccountNumber    = Clean([AccountNumberRaw]),
      AccountName      = Clean([AccountNameRaw]),
      City             = Clean([CityRaw]),
      State            = Clean([StateRaw]),
      ZIP              = "",
      RawProduct       = Clean([ProductRaw]),
      Volume           = Num([VolumeRaw]),
      GrossSpend       = Num([GrossRaw]),
      NetSpend         = Num([NetRaw]),
      Incentive        = Num([IncentiveRaw]),
      BilledWeightLbs  = Num([WeightRaw])
    ]),
    Out = Table.FromRecords(Table.Column(Shaped, "Rec"))
  in
    Out,

// --- Schema B: March / April "VnS" layout -----
// Column2 Sub-Parent Number  Column3 Sub-Parent Name
// Column4 Account Number     Column5 Account Name
// Column7 City               Column8 State          Column9 ZIP
// Column10 Product           Column11 Volume
// Column12 Package Billed Weight Lbs
// Column14 Gross Spend       Column15 Net Spend
ReadSchemaB = (sheet as table, monthNo as number) as table =>
  let
    Rows = Table.Skip(sheet, 5),
    Pick = Table.SelectColumns(Rows, {
      "Column2", "Column3", "Column4", "Column5", "Column7", "Column8",
      "Column9", "Column10", "Column11", "Column12", "Column14", "Column15" }),
    Named = Table.RenameColumns(Pick, {
      {"Column2", "SubNumberRaw"}, {"Column3", "SubNameRaw"},
      {"Column4", "AccountNumberRaw"}, {"Column5", "AccountNameRaw"},
      {"Column7", "CityRaw"}, {"Column8", "StateRaw"},
      {"Column9", "ZipRaw"}, {"Column10", "ProductRaw"},
      {"Column11", "VolumeRaw"}, {"Column12", "WeightRaw"},
      {"Column14", "GrossRaw"}, {"Column15", "NetRaw"} }),
    Kept = Table.SelectRows(Named, each
      KeepRow([SubNumberRaw], [AccountNumberRaw])
      and Clean([ProductRaw]) <> ""),
    Shaped = Table.AddColumn(Kept, "Rec", each [
      MonthNumber      = monthNo,
      SubNumber        = Clean([SubNumberRaw]),
      SubName          = SubNameOf([SubNumberRaw], [SubNameRaw]),
      IsMaskedSubParent = MaskedOf([SubNumberRaw], [SubNameRaw]),
      AccountNumber    = Clean([AccountNumberRaw]),
      AccountName      = Clean([AccountNameRaw]),
      City             = Clean([CityRaw]),
      State            = Clean([StateRaw]),
      ZIP              = Clean([ZipRaw]),
      RawProduct       = Clean([ProductRaw]),
      Volume           = Num([VolumeRaw]),
      GrossSpend       = Num([GrossRaw]),
      NetSpend         = Num([NetRaw]),
      // Schema B has no incentive column; derive it.
      Incentive        = Num([GrossRaw]) - Num([NetRaw]),
      BilledWeightLbs  = Num([WeightRaw])
    ]),
    Out = Table.FromRecords(Table.Column(Shaped, "Rec"))
  
```

```

in
    Out,

// --- the four months -----
Jan = ReadSchemaA("UPS_2025_01_VolumeSpend.xlsx", 1),
// Month 2 ONLY. This file also contains January - see the banner note.
Feb = ReadSchemaA("UPS_2025_02_VolumeSpend_YTD.xlsx", 2),
Mar = ReadSchemaB(
    Excel.Workbook(File.Contents(p_Folder & "\UPS_2025_03_AllTabs.xlsx"), null, true)
    {[Item = "VnS", Kind = "Sheet"]}[Data], 3),
Apr = ReadSchemaB(FirstSheet("UPS_2025_04_VolumeSpend.xlsx"), 4),

Combined = Table.Combine({Jan, Feb, Mar, Apr}),

// --- keys and derived columns -----
WithKeys = Table.AddColumn(
    Table.AddColumn(
        Table.AddColumn(
            Table.AddColumn(Combined, "Year", each 2025, Int64.Type),
            "MonthKey", each 2025 * 100 + [MonthNumber], Int64.Type),
            "AccountKey", each [SubNumber] & "|" & [AccountNumber], type text),
            "Quarter", each "Q" & Text.From(Number.RoundUp([MonthNumber] / 3)), type text),
    WithDate = Table.AddColumn(WithKeys, "DateKey",
        each #date(2025, [MonthNumber], 1), type date),

    WithMonthName = Table.AddColumn(WithDate, "MonthName",
        each Date.ToText(#date(2025, [MonthNumber], 1), "MMM"), type text),

// --- service mapping -----
// Join to the VnS half of the bridge. An unmapped product routes to a review
// bucket rather than silently inheriting a neighbouring service group.
Joined = Table.NestedJoin(WithMonthName, {"RawProduct"},
    Table.SelectRows(Dim_ServiceMapping, each [SourceSystem] = "VnS"), {"RawProduct"},
    "Map", JoinKind.LeftOuter),
Expanded = Table.ExpandTableColumn(Joined, "Map",
    {"ServiceGroup", "ServiceGroupShort", "ServiceTier", "Scope",
    "IncludeInRanking"}),
Defaults = Table.TransformColumns(Expanded, {
    {"ServiceGroup", each if _ = null then "Unmapped - Review" else _, type text},
    {"ServiceGroupShort", each if _ = null then "Unmapped" else _, type text},
    {"ServiceTier", each if _ = null then "Unmapped" else _, type text},
    {"Scope", each if _ = null then "Unknown" else _, type text},
    {"IncludeInRanking", each if _ = null then "No" else _, type text}},

// --- row-level quality flags -----
// Kept on the fact so the Data Quality page can quantify what the executive
// visuals exclude, rather than the exclusion being invisible in a filter pane.
Flags = Table.AddColumn(
    Table.AddColumn(
        Table.AddColumn(Defaults, "HasVolume",
            each if [Volume] > 0 then "Yes" else "No", type text),
            "IsSpendOnlyAdjustment",
            each if [Volume] <= 0 and Number.Abs([NetSpend]) > 0 then "Yes" else "No", type text),
            "IsNegativeSpend", each if [NetSpend] < 0 then "Yes" else "No", type text),
    Typed = Table.TransformColumnTypes(Flags, {
        {"MonthNumber", Int64.Type}, {"SubNumber", type text}, {"SubName", type text},
        {"AccountNumber", type text}, {"AccountName", type text},
        {"City", type text}, {"State", type text}, {"ZIP", type text},
        {"RawProduct", type text}, {"Volume", type number},
        {"GrossSpend", Currency.Type}, {"NetSpend", Currency.Type},
        {"Incentive", Currency.Type}, {"BilledWeightLbs", type number}})
in
    Typed

```

Query 4 — Fact_TimeInTransit

```

// =====
// 4. QUERY Fact_TimeInTransit
// =====
// New Source > Blank Query > rename to Fact_TimeInTransit > Advanced Editor.
//
// Reads the "TnT" sheet of the March file. This is the source of the 40%
// reliability weight in the recommendation score, so two rules matter:
//
// 1. The extract supplies an "On Time %" column (Column15). It is
//    DELIBERATELY NOT LOADED. Loading it invites someone to drop it into a
//    visual, where it gets averaged across rows and produces a number that is
//    wrong in a way nobody notices - averaging that column across the March
//    file returns about 98.2% against a true 96.46%, because a 4-package
//    service is weighted equally with a 47,916-package one. Only the counts
//    are loaded; the percentage is computed in DAX as a weighted ratio.
//
// 2. This extract is a SHIPPER VIEW population. Fact_VolumeSpend is a PAYOR
//    VIEW population. Packages Measured and Volume are close but not equal
//    (67,091 vs 68,527 for March) and must never be used as the same
//    denominator. ViewType carries the distinction onto the fact.
//
// Column offsets, verified against the real extract:
// Column2 Sub-Parent Number Column3 Sub-Parent Name
// Column4 Account Number Column5 Account Name
// Column7 City Column8 State Column9 ZIP
// Column10 Product Column11 Packages Measured
// Column12 Late by Time Column13 Late by Day Column14 Total Late
//
// When Time-in-Transit files arrive for other months, copy the Mar step, point
// it at that file, change the month number, and add it to Table.Combine.
// =====
let

```

```

Clean = (v) as text =>
  let
    t = if v = null then "" else Text.Trim(Text.From(v))
  in
    Text.Trim(if Text.StartsWith(t, "") then Text.RemoveRange(t, 0, 1) else t),

Num = (v) as number => if v = null then 0 else (try Number.From(v) otherwise 0),

KeepRow = (sub, acct) as logical =>
  let
    s = Text.Lower(Clean(sub)),
    a = Text.Lower(Clean(acct))
  in
    not (s = "grand total" or s = "total" or a = "grand total" or a = "total")
    and not (a = "" and s = ""),

SubNameOf = (numv, namev) as text =>
  if Clean(numv) = "@@" or Clean(namev) = "@@"
  then "Unassigned Sub-Parent"
  else Clean(namev),

ReadTnT = (fileName as text, monthNo as number) as table =>
  let
    Sheet = Excel.Workbook(File.Contents(p_Folder & "\ " & fileName), null, true)
    {Item = "TnT", Kind = "Sheet"}[Data],
    Rows = Table.Skip(Sheet, 5),
    Pick = Table.SelectColumns(Rows, {
      "Column2", "Column3", "Column4", "Column5", "Column7", "Column8",
      "Column9", "Column10", "Column11", "Column12", "Column13", "Column14"}),
    Named = Table.RenameColumns(Pick, {
      {"Column2", "SubNumberRaw"}, {"Column3", "SubNameRaw"},
      {"Column4", "AccountNumberRaw"}, {"Column5", "AccountNameRaw"},
      {"Column7", "CityRaw"}, {"Column8", "StateRaw"}, {"Column9", "ZipRaw"},
      {"Column10", "ProductRaw"}, {"Column11", "MeasuredRaw"},
      {"Column12", "LateTimeRaw"}, {"Column13", "LateDayRaw"},
      {"Column14", "TotalLateRaw"}},
    Kept = Table.SelectRows(Named, each
      KeepRow([SubNumberRaw], [AccountNumberRaw]
        and Clean([ProductRaw]) <> ""),
    Shaped = Table.AddColumn(Kept, "Rec", each
      let
        LateTime = Num([LateTimeRaw]),
        LateDay = Num([LateDayRaw]),
        // Trust the components over the supplied total. Where the
        // total is absent but the parts are present, rebuild it.
        Total = if Num([TotalLateRaw]) = 0 and (LateTime + LateDay) > 0
          then LateTime + LateDay
          else Num([TotalLateRaw]),
        Measured = Num([MeasuredRaw])
      in [
        MonthNumber = monthNo,
        SubNumber = Clean([SubNumberRaw]),
        SubName = SubNameOf([SubNumberRaw], [SubNameRaw]),
        AccountNumber = Clean([AccountNumberRaw]),
        AccountName = Clean([AccountNameRaw]),
        City = Clean([CityRaw]),
        State = Clean([StateRaw]),
        ZIP = Clean([ZipRaw]),
        RawProduct = Clean([ProductRaw]),
        PackagesMeasured = Measured,
        LateByTime = LateTime,
        LateByDay = LateDay,
        TotalLate = Total,
        OnTimePackages = Measured - Total,
        ViewType = "Shipper View"
      ]),
    Out = Table.FromRecords(Table.Column(Shaped, "Rec"))
  in
    Out,

Mar = ReadTnT("UPS_2025_03_AllTabs.xlsx", 3),

Combined = Table.Combine({Mar}),

WithKeys = Table.AddColumn(
  Table.AddColumn(
    Table.AddColumn(
      Table.AddColumn(Combined, "Year", each 2025, Int64.Type),
      "MonthKey", each 2025 * 100 + [MonthNumber], Int64.Type),
    "AccountKey", each [SubNumber] & "|" & [AccountNumber], type text),
  "Quarter", each "Q" & Text.From(Number.RoundUp([MonthNumber] / 3)), type text),

WithDate = Table.AddColumn(WithKeys, "DateKey",
  each #date(2025, [MonthNumber], 1), type date),
WithMonthName = Table.AddColumn(WithDate, "MonthName",
  each Date.ToText(#date(2025, [MonthNumber], 1), "MMM"), type text),

// Join on the TnT half of the bridge. This extract names services completely
// differently from Volume & Net Spend ("NEXT DAY AIR" vs
// "NDA REG / EXPRESS PKG"), which is exactly why the mapping table is keyed
// on (SourceSystem, RawProduct) rather than being a simple lookup.
Joined = Table.NestedJoin(WithMonthName, {"RawProduct"},
  Table.SelectRows(Dim_ServiceMapping, each [SourceSystem] = "TnT"), {"RawProduct"},
  "Map", JoinKind.LeftOuter),
Expanded = Table.ExpandTableColumn(Joined, "Map",
  {"ServiceGroup", "ServiceGroupShort", "ServiceTier", "Scope",
    "IncludeInRanking"}),
Defaults = Table.TransformColumns(Expanded, {
  {"ServiceGroup", each if _ = null then "Unmapped - Review" else _, type text},
  {"ServiceGroupShort", each if _ = null then "Unmapped" else _, type text},
  {"ServiceTier", each if _ = null then "Unmapped" else _, type text},
  {"Scope", each if _ = null then "Unknown" else _, type text},
  {"IncludeInRanking", each if _ = null then "No" else _, type text}},

```



```

Typed = Table.TransformColumnTypes(Defaults, {
    {"MonthNumber", Int64.Type}, {"SubNumber", type text}, {"SubName", type text},
    {"AccountNumber", type text}, {"AccountName", type text},
    {"City", type text}, {"State", type text}, {"ZIP", type text},
    {"RawProduct", type text}, {"PackagesMeasured", type number},
    {"LateByTime", type number}, {"LateByDay", type number},
    {"TotalLate", type number}, {"OnTimePackages", type number}})
in
Typed

```

Query 5 — Fact_Claims

```

// =====
// 5. QUERY Fact_Claims
// =====
// New Source > Blank Query > rename to Fact_Claims > Advanced Editor.
//
// NOTE THE MISSING COLUMN. This extract carries Sub-Parent, Account and Claims
// Category - and no product, service, or tracking number. There is therefore no
// join path from this table to the service dimension, and no way to state a
// loss claim rate "by service" from this source.
//
// The recommendation framework assigns 30% of its weight to Loss Claim Rate and
// 20% to exceptions. Neither can be allocated at service grain until UPS adds a
// service or tracking field to this export.
//
// Do NOT create a relationship from this table to Dim_Service - not a
// many-to-many through Dim_Account, not a bi-directional filter. Filtering
// claims by service through the account path returns "claims for accounts that
// used this service" - for an account using five services, the same claim count
// five times over - and any reader will take it as claims caused by that
// service. The model leaves the join absent so a service slicer visibly does
// nothing on the claims page. A blank is a correct answer to an unanswerable
// question; a plausible wrong number is not.
//
// Column offsets, verified against the real extract:
// Column2 Sub-Parent Number      Column3 Sub-Parent Name
// Column4 Account Number        Column5 Account Name
// Column6 Claims Category
// Column7 Issued Claims Qty      Column8 Issued Claims Pkg Qty
// Column10 Paid Claims Qty       Column11 Paid Claims Pkg Qty
// Column13 Registered Qty        Column14 Registered Pkg Qty
// Column16 Paid Claims Amount     Column17 Claims Declared Value
// =====

let
    Clean = (v) as text =>
        let
            t = if v = null then "" else Text.Trim(Text.From(v))
        in
            Text.Trim(if Text.StartsWith(t, "") then Text.RemoveRange(t, 0, 1) else t),

    Num = (v) as number => if v = null then 0 else (try Number.From(v) otherwise 0),

    KeepRow = (sub, acct) as logical =>
        let
            s = Text.Lower(Clean(sub)),
            a = Text.Lower(Clean(acct))
        in
            not (s = "grand total" or s = "total" or a = "grand total" or a = "total")
            and not (a = "" and s = ""),

    SubNameOf = (numv, namev) as text =>
        if Clean(numv) = "@@" or Clean(namev) = "@@"
        then "Unassigned Sub-Parent"
        else Clean(namev),

    ReadClaims = (fileName as text, monthNo as number) as table =>
        let
            Sheet = Excel.Workbook(File.Contents(p_Folder & "\" & fileName), null, true)
                {Item = "Claims", Kind = "Sheet"}[Data],
            Rows = Table.Skip(Sheet, 5),
            Pick = Table.SelectColumns(Rows, {
                "Column2", "Column3", "Column4", "Column5", "Column6", "Column7",
                "Column8", "Column10", "Column11", "Column13", "Column14",
                "Column16", "Column17"}),
            Named = Table.RenameColumns(Pick, {
                {"Column2", "SubNumberRaw"}, {"Column3", "SubNameRaw"},
                {"Column4", "AccountNumberRaw"}, {"Column5", "AccountNameRaw"},
                {"Column6", "CategoryRaw"}, {"Column7", "IssuedQtyRaw"},
                {"Column8", "IssuedPkgRaw"}, {"Column10", "PaidQtyRaw"},
                {"Column11", "PaidPkgRaw"}, {"Column13", "RegQtyRaw"},
                {"Column14", "RegPkgRaw"}, {"Column16", "PaidAmtRaw"},
                {"Column17", "DeclaredRaw"}}),
            // Blank category rows are report padding, not claim activity.
            Kept = Table.SelectRows(Named, each
                KeepRow([SubNumberRaw], [AccountNumberRaw])
                and Clean([CategoryRaw]) <> ""),
            Shaped = Table.AddColumn(Kept, "Rec", each [
                MonthNumber      = monthNo,
                SubNumber        = Clean([SubNumberRaw]),
                SubName           = SubNameOf([SubNumberRaw], [SubNameRaw]),
                AccountNumber     = Clean([AccountNumberRaw]),
                AccountName       = Clean([AccountNameRaw]),
                ClaimsCategory    = Clean([CategoryRaw]),
                IsLoss             = if Text.Lower(Clean([CategoryRaw])) = "loss"
                                    then "Yes" else "No",
                IssuedClaimsQty   = Num([IssuedQtyRaw]),
                IssuedClaimsPkgQty = Num([IssuedPkgRaw]),
                PaidClaimsQty     = Num([PaidQtyRaw]),
            ])
        in
            Shaped

```

```

        PaidClaimsPkgQty      = Num([PaidPkgRaw]),
        RegisteredClaimsQty   = Num([RegQtyRaw]),
        RegisteredClaimsPkgQty = Num([RegPkgRaw]),
        PaidClaimsAmount      = Num([PaidAmtRaw]),
        ClaimsDeclaredValue   = Num([DeclaredRaw]),
        // Stamped on every row so the limitation travels with the data
        // into any export, drillthrough or copied visual.
        ServiceAttribution   = "Not Available"
    }),
    Out = Table.FromRecords(Table.Column(Shaped, "Rec"))
in
    Out,

Mar = ReadClaims("UPS_2025_03_AllTabs.xlsx", 3),
Combined = Table.Combine({Mar}),

WithKeys = Table.AddColumn(
    Table.AddColumn(
        Table.AddColumn(
            Table.AddColumn(Combined, "Year", each 2025, Int64.Type),
            "MonthKey", each 2025 * 100 + [MonthNumber], Int64.Type),
            "AccountKey", each [SubNumber] & "|" & [AccountNumber], type text),
            "Quarter", each "Q" & Text.From(Number.RoundUp([MonthNumber] / 3)), type text),
    WithDate = Table.AddColumn(WithKeys, "DateKey",
        each #date(2025, [MonthNumber], 1), type date),
    WithMonthName = Table.AddColumn(WithDate, "MonthName",
        each Date.ToText(#date(2025, [MonthNumber], 1), "MMM"), type text),

Typed = Table.TransformColumnTypes(WithMonthName, {
    {"MonthNumber", Int64.Type}, {"SubNumber", type text}, {"SubName", type text},
    {"AccountNumber", type text}, {"AccountName", type text},
    {"ClaimsCategory", type text}, {"IsLoss", type text},
    {"IssuedClaimsQty", type number}, {"IssuedClaimsPkgQty", type number},
    {"PaidClaimsQty", type number}, {"PaidClaimsPkgQty", type number},
    {"RegisteredClaimsQty", type number}, {"RegisteredClaimsPkgQty", type number},
    {"PaidClaimsAmount", Currency.Type}, {"ClaimsDeclaredValue", Currency.Type}})
in
    Typed

```

Query 6 — Fact_Accessorial

```

// =====
// 6. QUERY Fact_Accessorial
// =====
// New Source > Blank Query > rename to Fact_Accessorial > Advanced Editor >
// paste everything below this banner (NOT the Fact_Claims query above).
//
// Not in the original build guide's data model. Added because it is the only
// table that explains WHY cost per piece moves. Net spend on its own says costs
// rose; this says the rise was fuel surcharge, address corrections or
// residential delivery - the difference between a carrier conversation and an
// internal process fix.
//
// Address Correction in particular is a pure self-inflicted cost: every unit is
// a package that shipped to a bad address the firm supplied. It is charged per
// package, it is fully avoidable through address validation at the point of
// shipment, and it appears nowhere in the Volume & Net Spend extract.
//
// Reconciliation guarantee: for a given month this table's NetSpend must sum to
// exactly the same value as Fact_VolumeSpend's NetSpend. For March both are
// $1,053,748.13. That equality is a free, strong integrity check and is wired to
// the [Reconciliation Status] measure on the Data Quality page.
//
// Column offsets:
// Column2 Sub-Parent Number   Column3 Sub-Parent Name
// Column4 Account Number     Column5 Account Name
// Column8 State               Column10 Product
// Column11 Freight Type Description Column12 Base or Accessorial Charge
// Column13 Number of Units    Column14 Accessorial Spend
// Column15 Gross Spend       Column16 Net Spend
// =====

let
    Clean = (v) as text =>
        let
            t = if v = null then "" else Text.Trim(Text.From(v))
        in
            Text.Trim(if Text.StartsWith(t, "") then Text.RemoveRange(t, 0, 1) else t),

    Num = (v) as number => if v = null then 0 else (try Number.From(v) otherwise 0),

    KeepRow = (sub, acct) as logical =>
        let
            s = Text.Lower(Clean(sub)),
            a = Text.Lower(Clean(acct))
        in
            not (s = "grand total" or s = "total" or a = "grand total" or a = "total")
            and not (a = "" and s = ""),

    SubNameOf = (numv, namev) as text =>
        if Clean(numv) = "@@" or Clean(namev) = "@@"
        then "Unassigned Sub-Parent"
        else Clean(namev),

    ReadAcc = (fileName as text, monthNo as number) as table =>
        let
            Sheet = Excel.Workbook(File.Contents(p_Folder & "\" & fileName), null, true)
                {[Item = "Accessorial", Kind = "Sheet"]}[Data],
            Rows = Table.Skip(Sheet, 5),
            Pick = Table.SelectColumns(Rows, {
                "Column2", "Column3", "Column4", "Column5", "Column8", "Column10",

```

```

        "Column11", "Column12", "Column13", "Column14", "Column15", "Column16"}),
    Named = Table.RenameColumns(Pick, {
        {"Column2", "SubNumberRaw"}, {"Column3", "SubNameRaw"},
        {"Column4", "AccountNumberRaw"}, {"Column5", "AccountNameRaw"},
        {"Column8", "StateRaw"}, {"Column10", "ProductRaw"},
        {"Column11", "FreightTypeRaw"}, {"Column12", "ChargeRaw"},
        {"Column13", "UnitsRaw"}, {"Column14", "AccSpendRaw"},
        {"Column15", "GrossRaw"}, {"Column16", "NetRaw"}}),
    Kept = Table.SelectRows(Named, each
        KeepRow([SubNumberRaw], [AccountNumberRaw])
        and Clean([ChargeRaw]) <> ""),
    Shaped = Table.AddColumn(Kept, "Rec", each [
        MonthNumber      = monthNo,
        SubNumber        = Clean([SubNumberRaw]),
        SubName           = SubNameOf([SubNumberRaw], [SubNameRaw]),
        AccountNumber     = Clean([AccountNumberRaw]),
        AccountName       = Clean([AccountNameRaw]),
        State             = Clean([StateRaw]),
        RawProduct        = Clean([ProductRaw]),
        FreightTypeDescription = Clean([FreightTypeRaw]),
        ChargeCategory    = Clean([ChargeRaw]),
        IsAccessorial      = if Text.Upper(Clean([FreightTypeRaw])) = "ACCESSORIAL"
                           then "Yes" else "No",
        NumberOfUnits     = Num([UnitsRaw]),
        AccessorialSpend  = Num([AccSpendRaw]),
        GrossSpend        = Num([GrossRaw]),
        NetSpend          = Num([NetRaw])
    ]),
    Out = Table.FromRecords(Table.Column(Shaped, "Rec"))
in
    Out,

Mar = ReadAcc("UPS_2025_03_AllTabs.xlsx", 3),
Combined = Table.Combine({Mar}),

WithKeys = Table.AddColumn(
    Table.AddColumn(
        Table.AddColumn(
            Table.AddColumn(Combined, "Year", each 2025, Int64.Type),
            "MonthKey", each 2025 * 100 + [MonthNumber], Int64.Type),
        "AccountKey", each [SubNumber] & "|" & [AccountNumber], type text),
        "Quarter", each "Q" & Text.From(Number.RoundUp([MonthNumber] / 3)), type text),
    WithDate = Table.AddColumn(WithKeys, "DateKey",
        each #date(2025, [MonthNumber], 1), type date),
    WithMonthName = Table.AddColumn(WithDate, "MonthName",
        each Date.ToText(#date(2025, [MonthNumber], 1), "MMM"), type text),

    Joined = Table.NestedJoin(WithMonthName, {"RawProduct"},
        Table.SelectRows(Dim_ServiceMapping, each [SourceSystem] = "VnS"), {"RawProduct"},
        "Map", JoinKind.LeftOuter),
    Expanded = Table.ExpandTableColumn(Joined, "Map",
        {"ServiceGroup", "ServiceGroupShort"}),
    Defaults = Table.TransformColumns(Expanded, {
        {"ServiceGroup", each if _ = null then "Unmapped - Review" else _, type text},
        {"ServiceGroupShort", each if _ = null then "Unmapped" else _, type text}},

    Typed = Table.TransformColumnTypes(Defaults, {
        {"MonthNumber", Int64.Type}, {"NumberOfUnits", type number},
        {"AccessorialSpend", Currency.Type}, {"GrossSpend", Currency.Type},
        {"NetSpend", Currency.Type}})
in
    Typed

```

Query 7 — Dim_Date

```

// =====
// 7. QUERY Dim_Date
// =====
// New Source > Blank Query > rename to Dim_Date > Advanced Editor.
//
// Generated for the full calendar year rather than derived from the data. A date
// table built from the months actually present collapses the axis: if a month is
// missing, a trend line joins the months either side of it and the gap becomes
// invisible. A full-year table shows the hole.
//
// IMPORTANT - do NOT mark this as a date table.
// It is MONTH grain: twelve rows, one per month, because every fact in this
// model is month grain. DAX time-intelligence functions (TOTALYTD,
// SAMEPERIODLASTYEAR, DATEADD) require a contiguous table of DAYS and will
// return blank or silently wrong values against twelve first-of-month rows. The
// measures in this build use MonthKey arithmetic instead, which is exact here.
//
// After loading: set the Sort by Column of MonthName and MonthYearLabel to
// SortOrder. Without that, every month axis renders alphabetically -
// Apr, Aug, Dec, Feb.
// =====

let
    ReportYear = 2025,
    // Months present in the data. Update as later months are added, or set to
    // {1..12} once the full year is loaded.
    LoadedMonths = {1, 2, 3, 4},
    // Anchors the completed-month flag. Change to a fixed date to reproduce a
    // previously published figure.
    AsOf = DateTime.Date(DateTime.LocalNow()),

    Rows = List.Transform({1 .. 12}, (m) =>
        let
            First = #date(ReportYear, m, 1),
            Last = Date.EndOfMonth(First),
            Completed = Last < AsOf,

```

```

        Loaded = List.Contains(LoadedMonths, m)
    in [
        DateKey      = First,
        Year         = ReportYear,
        MonthNumber  = m,
        MonthName    = Date.ToText(First, "MMM"),
        MonthNameLong = Date.ToText(First, "MMMM"),
        MonthKey     = ReportYear * 100 + m,
        MonthYearLabel = Date.ToText(First, "MMM yyyy"),
        Quarter      = "Q" & Text.From(Number.RoundUp(m / 3)),
        QuarterKey   = ReportYear * 10 + Number.RoundUp(m / 3),
        MonthEndDate = Last,
        IsCompletedMonth = if Completed then "Yes" else "No",
        IsLoaded     = if Loaded then "Yes" else "No",
        // The only flag that should gate a trend or annual comparison: the
        // month has both ended and actually been loaded.
        IsReportable = if Completed and Loaded then "Yes" else "No",
        SortOrder    = ReportYear * 100 + m
    ]),

Result = Table.FromRecords(Rows, type table [
    DateKey = date, Year = Int64.Type, MonthNumber = Int64.Type,
    MonthName = text, MonthNameLong = text, MonthKey = Int64.Type,
    MonthYearLabel = text, Quarter = text, QuarterKey = Int64.Type,
    MonthEndDate = date, IsCompletedMonth = text, IsLoaded = text,
    IsReportable = text, SortOrder = Int64.Type])
in
Result

```

Query 8 — Dim_Account

```

// =====
// 8. QUERY Dim_Account
// =====
// New Source > Blank Query > rename to Dim_Account > Advanced Editor.
//
// Built from the UNION of all three facts, not from Volume & Spend alone. An
// account can appear in the claims extract in a month where it has no billed
// volume; sourcing the dimension from one fact would drop it and its claims
// would land on a blank row.
// =====

let
    FromVolume = Table.SelectColumns(Fact_VolumeSpend,
        {"AccountKey", "SubNumber", "SubName", "AccountNumber", "AccountName",
        "City", "State", "ZIP"}),
    FromTransit = Table.SelectColumns(Fact_TimeInTransit,
        {"AccountKey", "SubNumber", "SubName", "AccountNumber", "AccountName",
        "City", "State", "ZIP"}),
    // The claims extract carries no geography; pad so the columns align.
    FromClaims = Table.AddColumn(
        Table.AddColumn(
            Table.AddColumn(
                Table.SelectColumns(Fact_Claims,
                    {"AccountKey", "SubNumber", "SubName", "AccountNumber", "AccountName"}),
                "City", each "", type text),
                "State", each "", type text),
                "ZIP", each "", type text),
        Combined = Table.Combine({FromVolume, FromTransit, FromClaims}),

    // Blank attributes must not win the deduplication. Sorting non-blank State
    // and City first means the surviving row carries the populated geography,
    // which the claims rows do not supply.
    Ranked = Table.Sort(Combined, {
        {"AccountKey", Order.Ascending},
        {each if [State] = null or [State] = "" then 1 else 0, Order.Ascending},
        {each if [City] = null or [City] = "" then 1 else 0, Order.Ascending}}),

    // Table.Buffer is required: without it the sort is not guaranteed to be
    // honoured by Table.Distinct and the wrong row can win.
    Distinct = Table.Distinct(Table.Buffer(Ranked), {"AccountKey"}),

    WithRegion = Table.AddColumn(Distinct, "Region", each
        let S = Text.Upper(if [State] = null then "" else [State]) in
        if List.Contains({"CT", "ME", "MA", "NH", "RI", "VT", "NJ", "NY", "PA"}, S) then "Northeast"
        else if List.Contains({"IL", "IN", "MI", "OH", "WI", "IA", "KS", "MN", "MO",
            "NE", "ND", "SD"}, S) then "Midwest"
        else if List.Contains({"DE", "FL", "GA", "MD", "NC", "SC", "VA", "DC", "WV", "AL", "KY", "MS",
            "TN", "AR", "LA", "OK", "TX"}, S) then "South"
        else if List.Contains({"AZ", "CO", "ID", "MT", "NV", "NM", "UT", "WY", "AK",
            "CA", "HI", "OR", "WA"}, S) then "West"
        else "Unknown / International", type text),

    // Coverage flags. Built from buffered key lists rather than a nested
    // Table.SelectRows per row: the nested form re-scans the whole fact table
    // for every account, and inside the inner lambda [AccountKey] is ambiguous
    // between the outer and inner row.
    TransitKeys = List.Buffer(List.Distinct(Table.Column(Fact_TimeInTransit, "AccountKey"))),
    ClaimsKeys = List.Buffer(List.Distinct(Table.Column(Fact_Claims, "AccountKey"))),

    WithFlags = Table.AddColumn(
        Table.AddColumn(WithRegion, "HasTransitData",
            each if List.Contains(TransitKeys, [AccountKey]) then "Yes" else "No", type text),
        "HasClaimsData",
            each if List.Contains(ClaimsKeys, [AccountKey]) then "Yes" else "No", type text),

    WithDisplay = Table.AddColumn(WithFlags, "AccountDisplay",
        each [AccountName] & " (" & [AccountNumber] & ")", type text),

    Sorted = Table.Sort(WithDisplay,

```

```

    {"SubName", Order.Ascending}, {"AccountName", Order.Ascending}})
in
    Sorted

```

Query 9 — Dim_Service

```

// =====
// 9. QUERY Dim_Service
// =====
// New Source > Blank Query > rename to Dim_Service > Advanced Editor.
//
// The de-duplicated service dimension the MODEL relates to. Dim_ServiceMapping
// has grain (SourceSystem, RawProduct) - 42 rows across two vocabularies - so
// RawProduct is not unique and it cannot be the one side of a relationship.
// This table has one row per Service Group and is what slicers bind to.
//
// After loading, hide Dim_ServiceMapping from the report view. It is needed for
// the Data Quality page's mapping inventory but must never appear in a slicer,
// and no measure may filter it - REMOVEFILTERS on a table with no relationship
// to the facts is a no-op that returns silently wrong numbers.
// =====

let
    Source = Dim_ServiceMapping,
    Trimmed = Table.SelectColumns(Source, {
        "ServiceGroup", "ServiceGroupShort", "ServiceTier", "Scope",
        "IncludeInRanking", "ServicePriority"}),
    Distinct = Table.Distinct(Trimmed, {"ServiceGroup"}),
    // A review bucket so an unmapped fact row finds a parent instead of becoming
    // a blank-key orphan on every visual.
    WithReview = Table.Combine({Distinct, #table(
        type table [ServiceGroup = text, ServiceGroupShort = text,
            ServiceTier = text,
            Scope = text, IncludeInRanking = text, ServicePriority = Int64.Type],
        {"Unmapped - Review", "Unmapped", "Unmapped", "Unknown", "No", 98})}),
    Deduped = Table.Distinct(WithReview, {"ServiceGroup"}),
    Sorted = Table.Sort(Deduped, {"ServicePriority", Order.Ascending})
in
    Sorted

```

Close and apply

Home > Close & Apply. All nine queries should load without error. If any fail, fix them before continuing — the model steps assume all nine exist.

4. Set up the model

4.1 Relationships

Model view. Drag from the **many** side to the **one** side. Every relationship is single-direction — *Cross filter direction: Single*.

From (many)	Column	To (one)	Column
Fact_VolumeSpend	MonthKey	Dim_Date	MonthKey
Fact_TimeInTransit	MonthKey	Dim_Date	MonthKey
Fact_Claims	MonthKey	Dim_Date	MonthKey
Fact_Accessorial	MonthKey	Dim_Date	MonthKey
Fact_VolumeSpend	AccountKey	Dim_Account	AccountKey
Fact_TimeInTransit	AccountKey	Dim_Account	AccountKey
Fact_Claims	AccountKey	Dim_Account	AccountKey
Fact_Accessorial	AccountKey	Dim_Account	AccountKey
Fact_VolumeSpend	ServiceGroup	Dim_Service	ServiceGroup
Fact_TimeInTransit	ServiceGroup	Dim_Service	ServiceGroup
Fact_Accessorial	ServiceGroup	Dim_Service	ServiceGroup

Do NOT connect Fact_Claims to Dim_Service

There is no key to join on — the claims extract has no product, service or tracking number. The tempting workaround is a many-to-many through Dim_Account, or a bi-directional filter. Both produce numbers; neither produces meaning. Filtering claims by service through the account path returns *claims for accounts that used this service* — for an account using five services, the same claim count five times over — and any reader will take it as claims caused by that service. **Leave the relationship out so a service slicer visibly does nothing on the claims page.** That absence is the honest signal.

No bi-directional relationships anywhere

With three fact tables at different grains, bi-directional filtering creates ambiguous filter paths that change measure results depending on which visual the user clicked first. Every relationship stays Single.

4.2 Sort the month columns

Data view, select Dim_Date, then for each of MonthName and MonthYearLabel:

1. Select the column.
2. **Column tools > Sort by column.**
3. Choose SortOrder.

This one is not optional

Without it every month axis in the report sorts alphabetically — Apr, Aug, Dec, Feb, Jan... Trend lines become meaningless, and it is easy to miss because the chart still looks like a chart. Check a month axis reads Jan, Feb, Mar before going further. It also silently resets if you retype a column, so re-check after any model edit.

4.3 Do NOT mark Dim_Date as a date table

Dim_Date is **month grain** — twelve rows, one per month — because every fact in this model is month grain. There is no daily shipment data to join to.

That makes the standard DAX time-intelligence functions unusable here. **TOTALYTD**, **SAMEPERIODLASTYEAR** and **DATEADD** all need a contiguous table of *days*; handed twelve first-of-month rows they return blank or silently wrong values rather than raising an error. The measures in Section 5 use MonthKey arithmetic instead, which is exact at this grain.

4.4 What-if parameters

Modeling > New parameter > Numeric range. These drive the qualification rules, and exposing them lets leadership move the most contestable assumption in the model during a review and watch the ranking respond.

Name	Min	Max	Increment	Default
Volume Threshold	0	10000	250	1000
Share Threshold	0	0.1	0.005	0.01

4.5 Hide what should not be sliced

Hide	Why
Dim_ServiceMapping (whole table)	It is the two-vocabulary bridge, needed by the Data Quality page but never as a slicer. No measure may filter it — it has no relationship to any fact, so REMOVEFILTERS on it is a no-op that returns silently wrong numbers.
Key columns — MonthKey, AccountKey, SortOrder, DateKey	Nothing good comes of a user dragging a surrogate key onto a visual.
Raw numeric fact columns	Expose measures, not columns, to the report view.

4.6 Apply the report theme (optional but recommended)

The theme is a JSON file. You cannot download it either, so paste it out of Appendix A at the back of this document into Notepad and save it as `UPS_Executive_Theme.json` — in the Save dialog set *Save as type* to **All Files** so Notepad does not append `.txt`.

Then: **View > Themes > Browse for themes** and pick the file.

It carries a colour palette validated for colour-vision deficiency, hairline gridlines, data labels off by default, and — deliberately — no dual-axis configuration. Skip it if you are short of time; nothing else depends on it.

4.7 Create the measures table

Home > Enter data, one column, one row, name the table `_Measures`, load it, then hide its column. Every measure in Section 5 goes here, which keeps the field list split cleanly between things you slice by and things you measure.

5. The measures

112 measures. Select `_Measures` in the Fields pane, then for each one: **Home > New measure**, paste the whole block including the name and the `=`, press Enter.

They are grouped as they are in the source files. Create them in order — later measures reference earlier ones, and Power BI will show an error until the dependency exists.

Set the display folder as you go

With the measure selected, use the **Measure tools** ribbon to set *Display folder* to the group name shown below. 112 measures in a flat list is unusable; in five folders it is navigable.

Group 1 — Volume, Spend and Cost (21 measures)

Total Shipments

```
Total Shipments =
SUM ( Fact_VolumeSpend[Volume] )
```

Rankable Shipments

```
Rankable Shipments =
CALCULATE (
    [Total Shipments],
    KEEPFILTERS ( Fact_VolumeSpend[IncludeInRanking] = "Yes" ),
    KEEPFILTERS ( Fact_VolumeSpend[HasVolume] = "Yes" )
)
```

YTD Shipments

```
YTD Shipments =
VAR CurrentYear = MAX ( Dim_Date[Year] )
VAR CurrentMonth = MAX ( Dim_Date[MonthNumber] )
RETURN
    CALCULATE (
        [Total Shipments],
        REMOVEFILTERS ( Dim_Date ),
        Dim_Date[Year] = CurrentYear,
        Dim_Date[MonthNumber] <= CurrentMonth
    )
```

Shipments PY

```
Shipments PY =
VAR PriorKeys =
    SELECTCOLUMNS ( VALUES ( Dim_Date[MonthKey] ), "@Key", Dim_Date[MonthKey] - 100 )
RETURN
    CALCULATE (
        [Total Shipments],
        REMOVEFILTERS ( Dim_Date ),
        TREATAS ( PriorKeys, Dim_Date[MonthKey] )
    )
```

Shipments YoY %

```
Shipments YoY % =
VAR Current = [Total Shipments]
VAR Prior = [Shipments PY]
RETURN
    DIVIDE ( Current - Prior, Prior )
```

Shipments LY (Source)

```
Shipments LY (Source) =
SUM ( Fact_VolumeSpend[VolumeLY] )
```

Total Net Spend

```
Total Net Spend =
SUM ( Fact_VolumeSpend[NetSpend] )
```

Total Gross Spend

```
Total Gross Spend =
SUM ( Fact_VolumeSpend[GrossSpend] )
```

Total Incentive

```
Total Incentive =
SUM ( Fact_VolumeSpend[Incentive] )
```

Incentive %

```
Incentive % =
DIVIDE ( [Total Incentive], [Total Gross Spend] )
```


YTD Net Spend

```

YTD Net Spend =
VAR CurrentYear = MAX ( Dim_Date[Year] )
VAR CurrentMonth = MAX ( Dim_Date[MonthNumber] )
RETURN
    CALCULATE (
        [Total Net Spend],
        REMOVEFILTERS ( Dim_Date ),
        Dim_Date[Year] = CurrentYear,
        Dim_Date[MonthNumber] <= CurrentMonth
    )

```

Net Spend PY

```

Net Spend PY =
VAR PriorKeys =
    SELECTCOLUMNS ( VALUES ( Dim_Date[MonthKey] ), "@Key", Dim_Date[MonthKey] - 100 )
RETURN
    CALCULATE (
        [Total Net Spend],
        REMOVEFILTERS ( Dim_Date ),
        TREATAS ( PriorKeys, Dim_Date[MonthKey] )
    )

```

Net Spend YoY %

```

Net Spend YoY % =
DIVIDE ( [Total Net Spend] - [Net Spend PY], [Net Spend PY] )

```

Avg Net Spend per Shipment

```

Avg Net Spend per Shipment =
DIVIDE (
    CALCULATE (
        [Total Net Spend],
        KEEPFILTERS ( Fact_VolumeSpend[IncludeInRanking] = "Yes" ),
        KEEPFILTERS ( Fact_VolumeSpend[HasVolume] = "Yes" )
    ),
    [Rankable Shipments]
)

```

Total Billed Weight

```

Total Billed Weight =
CALCULATE (
    SUM ( Fact_VolumeSpend[BilledWeightLbs] ),
    KEEPFILTERS ( Fact_VolumeSpend[IncludeInRanking] = "Yes" ),
    KEEPFILTERS ( Fact_VolumeSpend[HasVolume] = "Yes" )
)

```

Avg Billed Weight per Shipment

```

Avg Billed Weight per Shipment =
DIVIDE ( [Total Billed Weight], [Rankable Shipments] )

```

Avg Net Spend per Billed Lb

```

Avg Net Spend per Billed Lb =
DIVIDE (
    CALCULATE (
        [Total Net Spend],
        KEEPFILTERS ( Fact_VolumeSpend[IncludeInRanking] = "Yes" ),
        KEEPFILTERS ( Fact_VolumeSpend[HasVolume] = "Yes" )
    ),
    [Total Billed Weight]
)

```

Weight-Adjusted Cost Premium

```

Weight-Adjusted Cost Premium =
VAR BlendedRatePerLb =
    CALCULATE (
        [Avg Net Spend per Billed Lb],
        REMOVEFILTERS ( Dim_Service ),
        VALUES ( Dim_Date[MonthKey] )
    )
VAR ExpectedSpend = [Total Billed Weight] * BlendedRatePerLb
RETURN
    DIVIDE ( [Total Net Spend] - ExpectedSpend, ExpectedSpend )

```

Service Volume Mix %

```

Service Volume Mix % =
DIVIDE (
    [Rankable Shipments],
    CALCULATE ( [Rankable Shipments], REMOVEFILTERS ( Dim_Service ) )
)

```

Service Spend Mix %

```

Service Spend Mix % =
DIVIDE (
    [Total Net Spend],
    CALCULATE ( [Total Net Spend], REMOVEFILTERS ( Dim_Service ) )
)

```

Account Volume Mix %

```
Account Volume Mix % =
DIVIDE (
    [Rankable Shipments],
    CALCULATE ( [Rankable Shipments], REMOVEFILTERS ( Dim_Account ) )
)
```

Group 2 — Reliability (21 measures)**Packages Measured**

```
Packages Measured =
SUM ( Fact_TimeInTransit[PackagesMeasured] )
```

Total Late

```
Total Late =
SUM ( Fact_TimeInTransit[TotalLate] )
```

Late by Day

```
Late by Day =
SUM ( Fact_TimeInTransit[LateByDay] )
```

Late by Time

```
Late by Time =
SUM ( Fact_TimeInTransit[LateByTime] )
```

On-Time Packages

```
On-Time Packages =
SUM ( Fact_TimeInTransit[OnTimePackages] )
```

On-Time Delivery %

```
On-Time Delivery % =
DIVIDE ( [On-Time Packages], [Packages Measured] )
```

Late Delivery %

```
Late Delivery % =
DIVIDE ( [Total Late], [Packages Measured] )
```

Late by Day %

```
Late by Day % =
DIVIDE ( [Late by Day], [Packages Measured] )
```

Late by Time %

```
Late by Time % =
DIVIDE ( [Late by Time], [Packages Measured] )
```

Severity Mix (Day Share of Late)

```
Severity Mix (Day Share of Late) =
DIVIDE ( [Late by Day], [Total Late] )
```

OTD % PM

```
OTD % PM =
VAR CurrentKey = SELECTEDVALUE ( Dim_Date[MonthKey] )
VAR PriorKey =
    IF ( MOD ( CurrentKey, 100 ) = 1, CurrentKey - 89, CurrentKey - 1 )
RETURN
    IF (
        NOT ISBLANK ( CurrentKey ),
        CALCULATE (
            [On-Time Delivery %],
            REMOVEFILTERS ( Dim_Date ),
            Dim_Date[MonthKey] = PriorKey
        )
    )
```

OTD % Delta MoM

```
OTD % Delta MoM =
VAR Current = [On-Time Delivery %]
VAR Prior = [OTD % PM]
RETURN
    IF ( NOT ISBLANK ( Current ) && NOT ISBLANK ( Prior ), Current - Prior )
```

OTD % Annual Benchmark

```
OTD % Annual Benchmark =
CALCULATE (
    [On-Time Delivery %],
    REMOVEFILTERS ( Dim_Date[MonthNumber], Dim_Date[MonthName], Dim_Date[Quarter],
        Dim_Date[MonthYearLabel], Dim_Date[MonthKey] )
)
```

OTD Volatility (pp)

```

OTD Volatility (pp) =
VAR MonthlySeries =
    ADDCOLUMNS (
        CALCULATETABLE (
            VALUES ( Dim_Date[MonthKey] ),
            Dim_Date[IsReportable] = "Yes"
        ),
        "@OTD", [On-Time Delivery %]
    )
VAR Clean = FILTER ( MonthlySeries, NOT ISBLANK ( [@OTD] ) )
RETURN
    IF ( COUNTROWS ( Clean ) >= 2, STDEVX.P ( Clean, [@OTD] ) * 100 )

```

Worst OTD Month

```

Worst OTD Month =
VAR MonthlySeries =
    ADDCOLUMNS (
        CALCULATETABLE (
            VALUES ( Dim_Date[MonthYearLabel] ),
            Dim_Date[IsReportable] = "Yes"
        ),
        "@OTD", [On-Time Delivery %]
    )
VAR Clean = FILTER ( MonthlySeries, NOT ISBLANK ( [@OTD] ) )
RETURN
    IF (
        NOT ISEMPTY ( Clean ),
        CONCATENATEX (
            TOPN ( 1, Clean, [@OTD], ASC ),
            Dim_Date[MonthYearLabel] & " (" & FORMAT ( [@OTD], "0.0%" ) & ")"
        )
    )

```

OTD Wilson Lower Bound

```

OTD Wilson Lower Bound =
VAR N = [Packages Measured]
VAR Successes = [On-Time Packages]
VAR Z = 1.96 // 95% two-sided
VAR P = DIVIDE ( Successes, N )
VAR Denominator = 1 + DIVIDE ( Z * Z, N )
VAR Centre = P + DIVIDE ( Z * Z, 2 * N )
VAR Margin =
    Z * SQRT ( DIVIDE ( P * ( 1 - P ), N ) + DIVIDE ( Z * Z, 4 * N * N ) )
RETURN
    IF ( N > 0, DIVIDE ( Centre - Margin, Denominator ) )

```

OTD Confidence Gap (pp)

```

OTD Confidence Gap (pp) =
VAR Gap = [On-Time Delivery %] - [OTD Wilson Lower Bound]
RETURN
    IF ( NOT ISBLANK ( Gap ), Gap * 100 )

```

OTD Confidence Label

```

OTD Confidence Label =
VAR Gap = [OTD Confidence Gap (pp)]
RETURN
    SWITCH (
        TRUE (),
        ISBLANK ( Gap ), "No transit data",
        Gap <= 0.5, "High confidence",
        Gap <= 2.0, "Moderate confidence",
        Gap <= 10.0, "Low confidence - small sample",
        "Not interpretable - sample too small"
    )

```

Months with Transit Data

```

Months with Transit Data =
CALCULATE (
    DISTINCTCOUNT ( Fact_TimeInTransit[MonthKey] ),
    REMOVEFILTERS ( Dim_Service, Dim_Account )
)

```

Transit Coverage %

```

Transit Coverage % =
DIVIDE (
    CALCULATE ( DISTINCTCOUNT ( Fact_TimeInTransit[MonthKey] ) ),
    CALCULATE (
        DISTINCTCOUNT ( Fact_VolumeSpend[MonthKey] ),
        REMOVEFILTERS ( Dim_Service )
    )
)

```

Has Transit Data

```

Has Transit Data =
IF ( [Packages Measured] > 0, "Yes", "No" )

```

Group 3 — Claims and Loss Risk (23 measures)

Issued Claims Pkgs

```
Issued Claims Pkgs =
SUM ( Fact_Claims[IssuedClaimsPkgQty] )
```

Paid Claims Pkgs

```
Paid Claims Pkgs =
SUM ( Fact_Claims[PaidClaimsPkgQty] )
```

Registered Claims Pkgs

```
Registered Claims Pkgs =
SUM ( Fact_Claims[RegisteredClaimsPkgQty] )
```

Paid Claims Amount

```
Paid Claims Amount =
SUM ( Fact_Claims[PaidClaimsAmount] )
```

Claims Declared Value

```
Claims Declared Value =
SUM ( Fact_Claims[ClaimsDeclaredValue] )
```

Loss Claims

```
Loss Claims =
CALCULATE (
    [Issued Claims Pkgs],
    KEEPFILTERS ( Fact_Claims[ClaimsCategory] = "Loss" )
)
```

Damage Claims

```
Damage Claims =
CALCULATE (
    [Issued Claims Pkgs],
    KEEPFILTERS ( Fact_Claims[ClaimsCategory] = "Damage" )
)
```

Loss Claims Paid Amount

```
Loss Claims Paid Amount =
CALCULATE (
    [Paid Claims Amount],
    KEEPFILTERS ( Fact_Claims[ClaimsCategory] = "Loss" )
)
```

Claims Recovery Rate

```
Claims Recovery Rate =
DIVIDE ( [Paid Claims Pkgs], [Issued Claims Pkgs] )
```

Avg Paid per Claim

```
Avg Paid per Claim =
DIVIDE ( [Paid Claims Amount], [Paid Claims Pkgs] )
```

Loss Claim Rate per 10k

```
Loss Claim Rate per 10k =
DIVIDE ( [Loss Claims], [Rankable Shipments] ) * 10000
```

Claims Rate per 10k

```
Claims Rate per 10k =
DIVIDE ( [Issued Claims Pkgs], [Rankable Shipments] ) * 10000
```

Loss Claim Rate

```
Loss Claim Rate =
DIVIDE ( [Loss Claims], [Rankable Shipments] )
```

Exception / Claims Rate

```
Exception / Claims Rate =
DIVIDE ( [Issued Claims Pkgs], [Rankable Shipments] )
```

Claims Cost per 1k Shipments

```
Claims Cost per 1k Shipments =
DIVIDE ( [Paid Claims Amount], [Rankable Shipments] ) * 1000
```

Firm-wide Loss Rate per 10k

```
Firm-wide Loss Rate per 10k =
CALCULATE (
    [Loss Claim Rate per 10k],
    REMOVEFILTERS ( Dim_Account, Dim_Service )
)
```

Loss Rate Index vs Firm

```
Loss Rate Index vs Firm =
DIVIDE ( [Loss Claim Rate per 10k], [Firm-wide Loss Rate per 10k] )
```

Loss Risk Flag

```
Loss Risk Flag =
VAR Idx = [Loss Rate Index vs Firm]
VAR Vol = [Rankable Shipments]
RETURN
  SWITCH (
    TRUE (),
    ISBLANK ( [Loss Claims] ) || [Loss Claims] = 0, "No loss activity",
    Vol < 500, "Insufficient volume to rate",
    Idx >= 3, "Critical - investigate",
    Idx >= 2, "Elevated - monitor",
    Idx >= 1, "Above firm average",
    "At or below firm average"
  )
```

Top Account Share of Loss Claims

```
Top Account Share of Loss Claims =
VAR ByAccount =
  ADDCOLUMNS (
    CALCULATETABLE ( VALUES ( Dim_Account[AccountKey] ), REMOVEFILTERS ( Dim_Account ) ),
    "@Loss", [Loss Claims]
  )
VAR TotalLoss = SUMX ( ByAccount, [@Loss] )
VAR TopLoss = MAXX ( ByAccount, [@Loss] )
RETURN
  DIVIDE ( TopLoss, TotalLoss )
```

Top Account by Loss Claims

```
Top Account by Loss Claims =
VAR ByAccount =
  ADDCOLUMNS (
    CALCULATETABLE ( VALUES ( Dim_Account[AccountName] ), REMOVEFILTERS ( Dim_Account ) ),
    "@Loss", [Loss Claims]
  )
RETURN
  CONCATENATEX ( TOPN ( 1, ByAccount, [@Loss], DESC ), Dim_Account[AccountName] )
```

Loss Claims (Allocated Proxy)

```
Loss Claims (Allocated Proxy) =
SUMX (
  VALUES ( Dim_Account[AccountKey] ),
  VAR AccountLoss = CALCULATE ( [Loss Claims], REMOVEFILTERS ( Dim_Service ) )
  VAR AccountVolumeAllServices =
    CALCULATE ( [Rankable Shipments], REMOVEFILTERS ( Dim_Service ) )
  VAR VolumeInScope = [Rankable Shipments]
  RETURN
    AccountLoss * DIVIDE ( VolumeInScope, AccountVolumeAllServices )
)
```

Loss Claim Rate per 10k (Allocated Proxy)

```
Loss Claim Rate per 10k (Allocated Proxy) =
DIVIDE ( [Loss Claims (Allocated Proxy)], [Rankable Shipments] ) * 10000
```

Claims Attribution Warning

```
Claims Attribution Warning =
"Allocated estimate. The claims extract carries no service or tracking key; "
& "account-level claims are distributed across services by shipment volume. "
& "Not a measured service-level loss rate. Do not use for service selection."
```

Group 4 — Scoring and Recommendation (22 measures)**Minimum Shipment Threshold**

```
Minimum Shipment Threshold =
SELECTEDVALUE ( 'Volume Threshold'[Volume Threshold], 1000 )
```

Minimum Volume Share

```
Minimum Volume Share =
SELECTEDVALUE ( 'Share Threshold'[Share Threshold], 0.01 )
```

Minimum Measured Packages

```
Minimum Measured Packages =
384
```

Service Volume Share

```
Service Volume Share =
DIVIDE (
  [Rankable Shipments],
  CALCULATE ( [Rankable Shipments], REMOVEFILTERS ( Dim_Service ) )
)
```

Annual Qualification Flag

```

Annual Qualification Flag =
VAR MeetsAbsolute = [Rankable Shipments] >= [Minimum Shipment Threshold]
VAR MeetsShare = [Service Volume Share] >= [Minimum Volume Share]
VAR MeetsMeasurement = [Packages Measured] >= [Minimum Measured Packages]

VAR IsRankableService = SELECTEDVALUE ( Dim_Service[IncludeInRanking] ) = "Yes"
RETURN
    IF ( MeetsAbsolute && MeetsShare && MeetsMeasurement && IsRankableService, 1, 0 )

```

Qualification Status

```

Qualification Status =
VAR Vol = [Rankable Shipments]
VAR Measured = [Packages Measured]
RETURN
    SWITCH (
        TRUE (),
        [Annual Qualification Flag] = 1, "Qualified",
        ISBLANK ( Vol ) || Vol = 0, "No shipment volume",
        Measured < [Minimum Measured Packages],
            "Transit sample too small (" & FORMAT ( Measured, "#,0" ) & " measured, "
            & FORMAT ( [Minimum Measured Packages], "#,0" ) & " required)",
        Vol < [Minimum Shipment Threshold],
            "Below volume floor (" & FORMAT ( Vol, "#,0" ) & " of "
            & FORMAT ( [Minimum Shipment Threshold], "#,0" ) & ")",
        [Service Volume Share] < [Minimum Volume Share],
            "Below " & FORMAT ( [Minimum Volume Share], "0.0%") & " volume share",
        "Excluded from ranking by service mapping"
    )

```

Qualified Service Count

```

Qualified Service Count =
COUNTROWS (
    FILTER (
        CALCULATETABLE (
            VALUES ( Dim_Service[ServiceGroup] ),
            REMOVEFILTERS ( Dim_Service )
        ),
        [Annual Qualification Flag] = 1
    )
)

```

Claims Service Key Available

```

Claims Service Key Available =
0

```

Score - Reliability

```

Score - Reliability =
VAR QualifiedSet =
    FILTER (
        CALCULATETABLE (
            VALUES ( Dim_Service[ServiceGroup] ),
            REMOVEFILTERS ( Dim_Service )
        ),
        [Annual Qualification Flag] = 1
    )
VAR WithMetric = ADDCOLUMNS ( QualifiedSet, "@M", [OTD Wilson Lower Bound] )
VAR Populated = FILTER ( WithMetric, NOT ISBLANK ( [@M] ) )
VAR MinV = MINX ( Populated, [@M] )
VAR MaxV = MAXX ( Populated, [@M] )
VAR Self = [OTD Wilson Lower Bound]
RETURN
    IF (
        [Annual Qualification Flag] = 1 && NOT ISBLANK ( Self ),
        IF ( MaxV = MinV, 1, DIVIDE ( Self - MinV, MaxV - MinV ) )
    )

```

Score - Loss Risk

```

Score - Loss Risk =
VAR QualifiedSet =
    FILTER (
        CALCULATETABLE (
            VALUES ( Dim_Service[ServiceGroup] ),
            REMOVEFILTERS ( Dim_Service )
        ),
        [Annual Qualification Flag] = 1
    )
VAR WithMetric = ADDCOLUMNS ( QualifiedSet, "@M", [Loss Claim Rate per 10k] )
VAR Populated = FILTER ( WithMetric, NOT ISBLANK ( [@M] ) )
VAR MinV = MINX ( Populated, [@M] )
VAR MaxV = MAXX ( Populated, [@M] )
VAR Self = [Loss Claim Rate per 10k]
RETURN
    IF (
        [Annual Qualification Flag] = 1 && NOT ISBLANK ( Self ) && [Claims Service Key Available] = 1,
        IF ( MaxV = MinV, 1, 1 - DIVIDE ( Self - MinV, MaxV - MinV ) )
    )

```

Score - Exception Risk

```

Score - Exception Risk =
VAR QualifiedSet =
    FILTER (
        CALCULATETABLE (
            VALUES ( Dim_Service[ServiceGroup] ),
            REMOVEFILTERS ( Dim_Service )
        ),
        [Annual Qualification Flag] = 1
    )
VAR WithMetric = ADDCOLUMNS ( QualifiedSet, "@M", [Claims Rate per 10k] )
VAR Populated = FILTER ( WithMetric, NOT ISBLANK ( [@M] ) )
VAR MinV = MINX ( Populated, [@M] )
VAR MaxV = MAXX ( Populated, [@M] )
VAR Self = [Claims Rate per 10k]
RETURN
    IF (
        [Annual Qualification Flag] = 1 && NOT ISBLANK ( Self ) && [Claims Service Key Available] = 1,
        IF ( MaxV = MinV, 1, 1 - DIVIDE ( Self - MinV, MaxV - MinV ) )
    )

```

Score - Cost

```

Score - Cost =
VAR QualifiedSet =
    FILTER (
        CALCULATETABLE (
            VALUES ( Dim_Service[ServiceGroup] ),
            REMOVEFILTERS ( Dim_Service )
        ),
        [Annual Qualification Flag] = 1
    )
VAR WithMetric = ADDCOLUMNS ( QualifiedSet, "@M", [Avg Net Spend per Billed Lb] )
VAR Populated = FILTER ( WithMetric, NOT ISBLANK ( [@M] ) )
VAR MinV = MINX ( Populated, [@M] )
VAR MaxV = MAXX ( Populated, [@M] )
VAR Self = [Avg Net Spend per Billed Lb]
RETURN
    IF (
        [Annual Qualification Flag] = 1 && NOT ISBLANK ( Self ),
        IF ( MaxV = MinV, 1, 1 - DIVIDE ( Self - MinV, MaxV - MinV ) )
    )

```

Weight - Reliability

```

Weight - Reliability =
0.40

```

Weight - Loss Risk

```

Weight - Loss Risk =
0.30

```

Weight - Exception Risk

```

Weight - Exception Risk =
0.20

```

Weight - Cost

```

Weight - Cost =
0.10

```

Effective Weight Total

```

Effective Weight Total =
VAR W =
    IF ( NOT ISBLANK ( [Score - Reliability] ), [Weight - Reliability], 0 )
    + IF ( NOT ISBLANK ( [Score - Loss Risk] ), [Weight - Loss Risk], 0 )
    + IF ( NOT ISBLANK ( [Score - Exception Risk] ), [Weight - Exception Risk], 0 )
    + IF ( NOT ISBLANK ( [Score - Cost] ), [Weight - Cost], 0 )
RETURN
    IF ( W > 0, W )

```

Service Consistency Score

```

Service Consistency Score =
VAR Weighted =
    IF ( NOT ISBLANK ( [Score - Reliability] ), [Score - Reliability] * [Weight - Reliability], 0 )
    + IF ( NOT ISBLANK ( [Score - Loss Risk] ), [Score - Loss Risk] * [Weight - Loss Risk], 0 )
    + IF ( NOT ISBLANK ( [Score - Exception Risk] ), [Score - Exception Risk] * [Weight - Exception Risk], 0 )
    + IF ( NOT ISBLANK ( [Score - Cost] ), [Score - Cost] * [Weight - Cost], 0 )
RETURN
    IF (
        [Annual Qualification Flag] = 1,
        DIVIDE ( Weighted, [Effective Weight Total] ) * 100
    )

```

Score Basis

```

Score Basis =
VAR Covered =
    IF (
        NOT ISBLANK ( [Score - Reliability] ),
        "Reliability " & FORMAT ( [Weight - Reliability] / [Effective Weight Total], "0%" ) & "; ",
        ""
    )
    & IF ( NOT ISBLANK ( [Score - Loss Risk] ), "Loss risk " & FORMAT ( [Weight - Loss Risk] / [Effective Weight Total], "0%" ) & "; ", "" )
    & IF ( NOT ISBLANK ( [Score - Exception Risk] ), "Exceptions " & FORMAT ( [Weight - Exception Risk] / [Effective Weight Total], "0%" ) & "; ", "" )
    & IF ( NOT ISBLANK ( [Score - Cost] ), "Cost " & FORMAT ( [Weight - Cost] / [Effective Weight Total], "0%" ), "" )
VAR Missing =
    IF ( ISBLANK ( [Score - Loss Risk] ) || ISBLANK ( [Score - Exception Risk] ),
        " | Claims components unavailable at service grain - weights reallocated.", "" )
RETURN
    "Effective weights: " & Covered & Missing

```

Service Rank

```

Service Rank =
IF (
    [Annual Qualification Flag] = 1,
    RANKX (
        FILTER (
            CALCULATETABLE (
                VALUES ( Dim_Service[ServiceGroup] ),
                REMOVEFILTERS ( Dim_Service )
            ),
            [Annual Qualification Flag] = 1
        ),
        [Service Consistency Score],
        ,
        DESC,
        DENSE
    )
)

```

Recommendation Category

```

Recommendation Category =
VAR Qualified = [Annual Qualification Flag] = 1
VAR QualCount = [Qualified Service Count]
VAR Tier = SELECTEDVALUE ( Dim_Service[ServiceTier] )
VAR Rank = [Service Rank]
VAR BestPremiumRank =
    RANKX (
        FILTER (
            CALCULATETABLE (
                VALUES ( Dim_Service[ServiceGroup] ),
                REMOVEFILTERS ( Dim_Service )
            ),
            [Annual Qualification Flag] = 1
            && SELECTEDVALUE ( Dim_Service[ServiceTier] ) = "Premium"
        ),
        [OTD Wilson Lower Bound],
        ,
        DESC,
        DENSE
    )
VAR IsVolumeAnchor = [Service Volume Share] >= 0.50
VAR Volatile = [OTD Volatility (pp)] > 1.5
VAR BelowFirmOTD = [On-Time Delivery %] < CALCULATE ( [On-Time Delivery %], REMOVEFILTERS ( Dim_Service ) )
RETURN
    SWITCH (
        TRUE (),
        NOT Qualified, "Not ranked - " & [Qualification Status],
        QualCount <= 1, "Single qualified service - no comparison possible",
        // The service the firm actually runs on. It is the default by
        // economics; the question the dashboard answers is whether its
        // reliability is acceptable, not whether to replace it.
        IsVolumeAnchor && BelowFirmOTD, "Volume anchor - reliability below firm average, monitor",
        IsVolumeAnchor, "Recommended default service",
        Rank = 1 && Tier <> "Premium", "Recommended default service",
        Rank = 1, "Best overall performer",
        BestPremiumRank = 1, "Recommended premium service",
        Volatile, "Service to monitor - inconsistent month to month",
        "Qualified alternative"
    )

```


Recommendation Callout

```

Recommendation Callout =
VAR MonthsCovered = [Months with Transit Data]
VAR BestService =
    CONCATENATEX (
        TOPN (
            1,
            FILTER (
                CALCULATETABLE (
                    VALUES ( Dim_Service[ServiceGroup] ),
                    REMOVEFILTERS ( Dim_Service )
                ),
                [Annual Qualification Flag] = 1
            ),
            [Service Consistency Score],
            DESC
        ),
        Dim_Service[ServiceGroup]
    )
VAR FirmOTD = CALCULATE ( [On-Time Delivery %], REMOVEFILTERS ( Dim_Service ) )
RETURN
    SWITCH (
        TRUE (),
        MonthsCovered = 0,
            "No Time-in-Transit data loaded. Reliability cannot be assessed and no service recommendation can be made.",
        MonthsCovered < 6,
            "PROVISIONAL - based on " & MonthsCovered & " month(s) of Time-in-Transit data. "
                & "Highest-scoring qualified service: " & BestService & ". "
                & "Firm-wide on-time delivery " & FORMAT ( FirmOTD, "0.0%" ) & " "
                & "Do not issue as an annual service policy until at least six completed months are loaded.",
            "Highest-scoring qualified service: " & BestService & ". Firm-wide on-time delivery "
                & FORMAT ( FirmOTD, "0.0%" ) & " across " & MonthsCovered & " months."
    )

```

Group 5 — Data Quality and Narrative (25 measures)**Months Loaded**

```

Months Loaded =
CALCULATE (
    DISTINCTCOUNT ( Dim_Date[MonthKey] ),
    Dim_Date[IsLoaded] = "Yes",
    REMOVEFILTERS ( Dim_Date )
)

```

Completed Months Loaded

```

Completed Months Loaded =
CALCULATE (
    DISTINCTCOUNT ( Dim_Date[MonthKey] ),
    Dim_Date[IsReportable] = "Yes",
    REMOVEFILTERS ( Dim_Date )
)

```

Missing Months

```

Missing Months =
VAR Expected =
    CALCULATE (
        DISTINCTCOUNT ( Dim_Date[MonthKey] ),
        Dim_Date[IsCompletedMonth] = "Yes",
        REMOVEFILTERS ( Dim_Date )
    )
RETURN
    Expected - [Completed Months Loaded]

```

Missing Months List

```

Missing Months List =
CONCATENATEX (
    CALCULATETABLE (
        VALUES ( Dim_Date[MonthYearLabel] ),
        Dim_Date[IsCompletedMonth] = "Yes",
        Dim_Date[IsLoaded] = "No",
        REMOVEFILTERS ( Dim_Date )
    ),
    Dim_Date[MonthYearLabel],
    ", ",
    Dim_Date[SortOrder],
    ASC
)

```

Annual Completeness %

```

Annual Completeness % =
DIVIDE ( [Months Loaded], 12 )

```

Volume Months Covered

```

Volume Months Covered =
CALCULATE ( DISTINCTCOUNT ( Fact_VolumeSpend[MonthKey] ), REMOVEFILTERS ( Dim_Date ) )

```

Transit Months Covered

```

Transit Months Covered =
CALCULATE ( DISTINCTCOUNT ( Fact_TimeInTransit[MonthKey] ), REMOVEFILTERS ( Dim_Date ) )

```

Claims Months Covered

```
Claims Months Covered =
CALCULATE ( DISTINCTCOUNT ( Fact_Claims[MonthKey] ), REMOVEFILTERS ( Dim_Date ) )
```

Coverage Summary

```
Coverage Summary =
"Volume & Spend: " & [Volume Months Covered] & " mo | "
& "Time-in-Transit: " & [Transit Months Covered] & " mo | "
& "Claims: " & [Claims Months Covered] & " mo"
```

Excluded Adjustment Spend

```
Excluded Adjustment Spend =
CALCULATE (
    SUM ( Fact_VolumeSpend[NetSpend] ),
    KEEPFILTERS ( Fact_VolumeSpend[IncludeInRanking] = "No" )
)
```

Excluded Adjustment Spend %

```
Excluded Adjustment Spend % =
DIVIDE (
    [Excluded Adjustment Spend],
    CALCULATE ( SUM ( Fact_VolumeSpend[NetSpend] ), REMOVEFILTERS ( Dim_Service ) )
)
```

Spend-Only Adjustment Rows

```
Spend-Only Adjustment Rows =
CALCULATE (
    COUNTROWS ( Fact_VolumeSpend ),
    KEEPFILTERS ( Fact_VolumeSpend[IsSpendOnlyAdjustment] = "Yes" )
)
```

Spend-Only Adjustment Value

```
Spend-Only Adjustment Value =
CALCULATE (
    SUM ( Fact_VolumeSpend[NetSpend] ),
    KEEPFILTERS ( Fact_VolumeSpend[IsSpendOnlyAdjustment] = "Yes" )
)
```

Unattributed Spend %

```
Unattributed Spend % =
DIVIDE (
    [Spend-Only Adjustment Value],
    CALCULATE ( SUM ( Fact_VolumeSpend[NetSpend] ), REMOVEFILTERS ( Dim_Service ) )
)
```

Unmapped Product Rows

```
Unmapped Product Rows =
CALCULATE (
    COUNTROWS ( Fact_VolumeSpend ),
    KEEPFILTERS ( Fact_VolumeSpend[ServiceGroup] = "Unmapped - Review" )
)
```

Masked Sub-Parent Volume

```
Masked Sub-Parent Volume =
CALCULATE (
    [Total Shipments],
    KEEPFILTERS ( Fact_VolumeSpend[IsMaskedSubParent] = "Yes" )
)
```

Accessorial Net Spend

```
Accessorial Net Spend =
SUM ( Fact_Accessorial[NetSpend] )
```

Spend Reconciliation Variance

```
Spend Reconciliation Variance =
VAR VolumeSpendTotal =
    CALCULATE ( SUM ( Fact_VolumeSpend[NetSpend] ), REMOVEFILTERS ( Dim_Service ) )
VAR AccessorialTotal =
    CALCULATE ( [Accessorial Net Spend], REMOVEFILTERS ( Dim_Service ) )
RETURN
    IF ( AccessorialTotal <> 0, VolumeSpendTotal - AccessorialTotal )
```

Reconciliation Status

```
Reconciliation Status =
VAR Variance = [Spend Reconciliation Variance]
RETURN
    SWITCH (
        TRUE (),
        ISBLANK ( Variance ), "No accessorial detail for this period",
        ABS ( Variance ) < 1, "Reconciled",
        ABS ( Variance ) < 100, "Rounding variance - " & FORMAT ( Variance, "$#,0.00" ),
        "VARIANCE - " & FORMAT ( Variance, "$#,0.00" ) & " - investigate before publishing"
    )
```

Measured vs Billed Coverage %

```
Measured vs Billed Coverage % =
DIVIDE ( [Packages Measured], [Rankable Shipments] )
```

Data Basis Banner

```
Data Basis Banner =
VAR VolMonths = [Volume Months Covered]
VAR TransitMonths = [Transit Months Covered]
VAR ClaimsMonths = [Claims Months Covered]
VAR Missing = [Missing Months List]
VAR Recon = [Reconciliation Status]
VAR Severity =
    SWITCH (
        TRUE (),
        TransitMonths = 0, "NO RELIABILITY DATA",
        TransitMonths < 6 || ClaimsMonths < 6, "PROVISIONAL",
        VolMonths < 12, "PARTIAL YEAR",
        "ANNUAL"
    )
RETURN
    Severity & " - " & [Coverage Summary]
    & IF ( LEN ( Missing ) > 0, " | Not yet loaded: " & Missing, "" )
    & IF ( Recon <> "Reconciled" && LEFT ( Recon, 2 ) = "VA", " | " & Recon, "" )
```

Data Basis Banner Colour

```
Data Basis Banner Colour =
SWITCH (
    TRUE (),
    [Transit Months Covered] = 0, "#B3261E",
    [Transit Months Covered] < 6 || [Claims Months Covered] < 6, "#B26B00",
    [Volume Months Covered] < 12, "#5B488A",
    "#1F6F54"
)
```

Title - Executive Overview

```
Title - Executive Overview =
VAR Period =
    IF (
        ISFILTERED ( Dim_Date[MonthYearLabel] ) || ISFILTERED ( Dim_Date[Quarter] ),
        CONCATENATEX (
            VALUES ( Dim_Date[MonthYearLabel] ),
            Dim_Date[MonthYearLabel], " ", Dim_Date[SortOrder], ASC
        ),
        SELECTEDVALUE ( Dim_Date[Year], "2025" ) & " full year"
    )
VAR Scope =
    IF (
        ISFILTERED ( Dim_Service[Scope] ),
        SELECTEDVALUE ( Dim_Service[Scope], "All scopes" ),
        "Domestic and international"
    )
RETURN
    "UPS Shipping Performance - " & Period & " - " & Scope
```

Title - Service Scorecard

```
Title - Service Scorecard =
"Service scorecard - " & [Qualified Service Count] & " of "
& CALCULATE (
    DISTINCTCOUNT ( Dim_Service[ServiceGroup] ),
    REMOVEFILTERS ( Dim_Service ),
    Dim_Service[IncludeInRanking] = "Yes"
)
& " services qualify at "
& FORMAT ( [Minimum Shipment Threshold], "#,0" ) & " shipments / "
& FORMAT ( [Minimum Volume Share], "0.0%" ) & " share"
```

Title - Claims

```
Title - Claims =
"Claims and loss risk - account level only ("
& [Claims Months Covered] & " month(s) loaded). "
& "Claims carry no service key."
```

Formatting

With each measure selected, set the format on the **Measure tools** ribbon:

Measure type	Format
Counts — shipments, packages, claims	Whole number, comma separated
Currency totals	Currency, 0 decimals
Cost per shipment / per claim	Currency, 2 decimals
Cost per billed lb	Currency, 3 decimals
On-time delivery, Wilson lower bound	Percentage, 2 decimals
Other percentages	Percentage, 1 decimal
Rates per 10k, index, score	Decimal, 1–2 places

4.8 Save

File > Save as, set *Save as type* to **Power BI files (*.pbix)**, and name it. Save again after each of the next sections — 112 measures is a lot of work to lose.

6. Build the report

Eight pages. The full visual-by-visual specification is in `powerbi/report_spec/00_page_layouts.md`; this is the working summary. Canvas 1280 × 720. Put a card bound to `[Data Basis Banner]` at the top of **every** page. A report that states its own reporting basis on every page cannot be misquoted from a screenshot.

Page	Contents
1. Executive Overview	5 KPI cards — Total Shipments, Total Net Spend, On-Time Delivery %, Loss Claim Rate per 10k, Avg Net Spend per Shipment. Line chart of On-Time Delivery % by MonthYearLabel. Card with [Recommendation Callout]. Table: ServiceGroup × Rankable Shipments, On-Time Delivery %, OTD Wilson Lower Bound, both cost views, Service Consistency Score, Qualification Status. Donut of volume mix.
2. Service Performance	Bar of On-Time Delivery % by service. Stacked column of Late by Day vs Late by Time by month — the severity split. Table with Qualification Status, OTD Confidence Label, OTD Volatility, Worst OTD Month.
3. Cost and Volume	Volume by month and net spend by month as two separate charts — never a dual axis. Paired bars: cost per shipment beside cost per billed pound. Cost bridge table with Avg Billed Weight per Shipment.
4. Claims and Exceptions	Pinned notice with [Claims Attribution Warning]. KPI row. Claims by category. Account concentration table sorted by Loss Rate Index vs Firm, with data bars. No service breakdown — that is deliberate.
5. Account and Location	Account scorecard. Volume by region. On-time by state. A table of accounts that have volume but no transit coverage — they are invisible in every reliability figure and need naming, not averaging away.
6. Annual Recommendation	Recommendation matrix. [Score Basis] card. The two threshold sliders. Score components. Remove the Month and Account slicers from this page — an annual policy sliced to one account in one month is not an annual policy.
7. Accessorial and Cost Drivers	Net spend by charge category. Monthly trend. Address Correction is the line to call out: every unit is a package sent to a bad address you supplied, charged per package, fully avoidable.
8. Data Quality	Coverage by month and source. Exclusion counts. Service mapping inventory (both vocabularies). Known limitations.

Two visual rules worth stating

Show both cost views side by side, always

Cost per shipment on its own is the single most misleading number in this data. Ground averages about 16 lb per piece against about 2 lb for 2nd Day Air, so per shipment they look almost identically priced — a package-mix artefact, not a rate. Per billed pound Ground is roughly seven times cheaper. A reader who sees only the per-shipment view will conclude that upgrading Ground volume to air is nearly free. Annotate the visual with that sentence; do not rely on the reader inferring it.

Set KPI cards to no interaction

Format > Edit interactions, set each card to None. The default is Highlight, and a user who clicks a card and sees the page change will assume the click meant something.

7. Verify before you publish

Seven checks. Each catches a specific failure mode, and each takes seconds. The first four matter most — they catch errors that produce numbers which are internally consistent and completely wrong.

#	Check	What it catches
1	January volume equals your January extract — not double it	The year-to-date overlap. The February file contains January too. Getting exactly double is the signature.
2	On-time delivery = 96.46% for March	Compare against the Grand Total row of the TnT sheet. A reading near 98.2% means a percentage is being averaged rather than weighted.
3	Page 8 Reconciliation Status = Reconciled	Accessorial detail must sum to net spend to the cent — \$1,053,748.13 for March. A variance means a source file is stale or partly loaded.
4	Service Volume Mix % sums to 100% across services	If it reads 100% on every row, a measure is filtering a table with no relationship to the facts and the REMOVEFILTERS is a no-op.
5	A month axis reads Jan → Dec	The sort-by columns in step 4.2 were not applied.
6	[YTD Shipments] and [OTD % PM] return values, not blanks	Time-intelligence functions used against the month-grain date table.
7	The banner reads PROVISIONAL	With one month of transit and claims data it must not say ANNUAL.

Reference figures from your data

Jan-Apr volume and spend; March transit, claims and accessorial:

Figure	Value
Total shipments	293,921
Total net spend	\$4,519,934
January volume	79,359
March net spend	\$1,053,748.13
March on-time delivery	96.46% across 67,091 measured packages
March loss claims	35, a rate of 5.1 per 10,000
Ground cost per billed lb	\$0.89 (vs \$6.48 for 2nd Day Air)

8. Troubleshooting

Error or symptom	Cause	Fix
We couldn't find the file	Wrong p_Folder, trailing backslash, or a file not renamed	Check the path in the error text — it shows exactly what was built
The column 'Column22' of the table wasn't found	The sheet has fewer columns than expected — usually the wrong file in the wrong name, e.g. a VnS file named as the January file	Confirm each renamed file is the layout the query expects
The key didn't match any rows in the table	A sheet name differs — the March file's tabs must be TnT, Claims, Accessorial, VnS	Open the file and check the tab names
Expression.Error: The name 'Dim_ServiceMapping' wasn't recognized	Queries created out of order, or a name typo	Names must match exactly; create query 2 before 3, 4 and 6
A measure shows a red squiggle	It references a measure you have not created yet	Create the measures in the order given in Section 5
January volume is exactly double	The February file was not filtered to month 2	Check the Feb step passes 2 to ReadSchemaA
Month axis alphabetical	Sort-by columns not set	Section 4.2
All visuals blank	Close & Apply not run, or relationships missing	Check the Model view shows 11 relationships

If the M does not work at all

These queries were written against your actual extracts and every column and row offset was verified by an independent implementation that reconciles to them exactly. But they have never been executed inside Power BI Desktop — there is no copy of Desktop in the environment they were written in. Treat the first run as a test.

If a query fails in a way this section does not cover, the fallback is to load each sheet manually — **Get Data > Excel**, pick the sheet, remove the top rows (7 for the January and February layout, 5 for all others), and rename columns to match the names used here. Slower, but the measures and the model structure are unaffected.

The one thing not to skip

Whatever route you take, verify check 1. The February file being year-to-date is the single trap in this data set that produces a plausible, confident, wrong answer — a 100% overstatement of January with no error message anywhere.

Appendix A — Report theme JSON

Paste into Notepad, save as `UPS_Executive_Theme.json` with *Save as type* set to **All Files**, then **View > Themes > Browse for themes**.

The categorical colours are assigned in a fixed order and never cycled. The order matters: it was chosen so that adjacent series stay distinguishable under the common forms of colour-vision deficiency. Do not reorder them.

```
{
  "name": "UPS 2025 Executive",
  "$schema": "https://raw.githubusercontent.com/microsoft/powerbi-desktop-samples/main/Report%20Theme%20JSON%20Schema/reportThemeSchema-2.114.json",

  "_comment_palette": "Categorical slots are assigned in fixed order and never cycled. Validated for colour-vision deficiency: worst all-pairs deutan separation 9.",

  "dataColors": [
    "#2a78d6",
    "#eb6834",
    "#1ba77a",
    "#eda100",
    "#e87ba4",
    "#008300",
    "#4a3aa7",
    "#e34948"
  ],

  "_comment_status": "Status colours are reserved. Never reuse one as a series colour - a bar that happens to be the fourth category must not read as 'critical'. A",
  "good": "#0ca30c",
  "neutral": "#898781",
  "bad": "#d03b3b",
  "maximum": "#0d366b",
  "center": "#f0efec",
  "minimum": "#cde2fb",
  "null": "#e1e0d9",

  "foreground": "#0b0b0b",
  "foregroundNeutralSecondary": "#52514e",
  "foregroundNeutralTertiary": "#898781",
  "background": "#fcfcfb",
  "backgroundLight": "#f9f9f7",
  "backgroundNeutral": "#e1e0d9",
  "tableAccent": "#2a78d6",
  "secondaryBackground": "#f0efec",
  "hyperLink": "#256abf",

  "textClasses": {
    "title": { "fontFace": "Segoe UI Semibold", "fontSize": 15, "color": "#0b0b0b" },
    "header": { "fontFace": "Segoe UI Semibold", "fontSize": 12, "color": "#0b0b0b" },
    "label": { "fontFace": "Segoe UI", "fontSize": 10, "color": "#52514e" },
    "callout": { "fontFace": "Segoe UI", "fontSize": 32, "color": "#0b0b0b" }
  },

  "visualStyles": {
    "****": {
      "background": [{ "show": true, "color": { "solid": { "color": "#fcfcfb" } }, "transparency": 0 }],
      "border": [{ "show": true, "color": { "solid": { "color": "#e1e0d9" } }, "radius": 6 }],
      "dropShadow": [{ "show": false }],
      "visualHeader": [{ "show": true, "background": { "solid": { "color": "#fcfcfb" } } }],
      "title": [{
        "show": true,
        "fontColor": { "solid": { "color": "#0b0b0b" } },
        "fontSize": 13,
        "fontFamily": "Segoe UI Semibold",
        "alignment": "left",
        "background": { "solid": { "color": "#fcfcfb" } }
      }],
      "subTitle": [{
        "show": true,
        "fontColor": { "solid": { "color": "#898781" } },
        "fontSize": 9,
        "alignment": "left"
      }],
    },

    "_comment_axes": "Gridlines are hairline, solid, one step off surface, and recessive. Never dashed - a dashed gridline competes with the data for attention",
    "categoryAxis": [{
      "show": true,
      "showAxisTitle": false,
      "labelColor": { "solid": { "color": "#898781" } },
      "fontSize": 9,
      "gridlineShow": false,
      "lineColor": { "solid": { "color": "#c3c2b7" } }
    }],
    "valueAxis": [{
      "show": true,
      "showAxisTitle": false,
      "labelColor": { "solid": { "color": "#898781" } },
      "fontSize": 9,
      "gridlineShow": true,
      "gridlineColor": { "solid": { "color": "#e1e0d9" } },
      "gridlineThickness": 1,
      "gridlineStyle": "solid",
      "_comment_secondary": "There is no secondary axis configuration in this theme, by design. A dual-axis chart makes two unrelated scales look comparable and",
    }],

    "legend": [{
      "show": true,
      "position": "TopLeft",
      "showTitle": false,
      "labelColor": { "solid": { "color": "#52514e" } },
      "fontSize": 9
    }],
  },
}
```



```

    },
    "labels": [{
      "show": false,
      "color": { "solid": { "color": "#52514e" } },
      "fontSize": 9,
      "_comment": "Data labels default OFF. Label selectively - the endpoint, the extreme, or the series the story is about. A number on every point goes unrea
    }]
  }
},
"card": {
  "**": {
    "labels": [{ "color": { "solid": { "color": "#0b0b0b" } }, "fontSize": 28, "fontFamily": "Segoe UI" }],
    "categoryLabels": [{ "show": true, "color": { "solid": { "color": "#898781" } }, "fontSize": 9 }],
    "background": [{ "show": true, "color": { "solid": { "color": "#fcfcfb" } } }],
    "border": [{ "show": true, "color": { "solid": { "color": "#e1e0d9" } }, "radius": 6 }]
  }
},
"columnChart": {
  "**": {
    "dataPoint": [{ "fill": { "solid": { "color": "#2a78d6" } } }],
    "_comment": "Single-series column charts use one hue, not the full categorical set. Reserve categorical colour for when the series themselves are the subje
  }
},
"barChart": {
  "**": {
    "dataPoint": [{ "fill": { "solid": { "color": "#2a78d6" } } }]
  }
},
"lineChart": {
  "**": {
    "lineStyles": [{ "strokeWidth": 2, "lineStyle": "solid", "showMarker": true, "markerSize": 5 }]
  }
},
"slicer": {
  "**": {
    "background": [{ "show": true, "color": { "solid": { "color": "#f9f9f7" } } }],
    "border": [{ "show": true, "color": { "solid": { "color": "#e1e0d9" } }, "radius": 6 }],
    "header": [{ "show": true, "fontColor": { "solid": { "color": "#52514e" } }, "fontSize": 9, "outline": "None" }],
    "items": [{ "fontColor": { "solid": { "color": "#0b0b0b" } }, "fontSize": 10 }],
    "selection": [{ "strictSingleSelect": false, "singleSelect": false }]
  }
},
"tableEx": {
  "**": {
    "grid": [{
      "gridVertical": false,
      "gridHorizontal": true,
      "gridHorizontalColor": { "solid": { "color": "#e1e0d9" } },
      "gridHorizontalWeight": 1,
      "rowPadding": 4,
      "outlineColor": { "solid": { "color": "#e1e0d9" } },
      "outlineWeight": 1
    }],
    "columnHeaders": [{
      "fontColor": { "solid": { "color": "#52514e" } },
      "backColor": { "solid": { "color": "#f9f9f7" } },
      "fontSize": 9,
      "bold": true,
      "alignment": "left"
    }],
    "values": [{
      "fontColor": { "solid": { "color": "#0b0b0b" } },
      "backColor": { "solid": { "color": "#fcfcfb" } },
      "fontSize": 10,
      "backColorSecondary": { "solid": { "color": "#fcfcfb" } },
      "_comment": "Banded rows are off. With a hairline horizontal rule and 4px padding, banding is redundant ink."
    }]
  }
},
"pivotTable": {
  "**": {
    "columnHeaders": [{ "fontColor": { "solid": { "color": "#52514e" } }, "backColor": { "solid": { "color": "#f9f9f7" } }, "fontSize": 9, "bold": true }],
    "values": [{ "fontColor": { "solid": { "color": "#0b0b0b" } }, "fontSize": 10 }]
  }
},
"scatterChart": {
  "**": {
    "_comment": "Cap scatter, bubble and map visuals at THREE series. Any pair of points can land adjacent on these forms, so the palette must clear the all-pa
    "fillPoint": [{ "show": true }],
    "bubbles": [{ "bubbleSize": -20 }]
  }
},
"actionButton": {
  "**": {
    "text": [{ "fontColor": { "solid": { "color": "#256abf" } }, "fontSize": 10 }],
    "outline": [{ "lineColor": { "solid": { "color": "#c3c2b7" } }, "transparency": 0, "weight": 1 }],
    "fill": [{ "show": true, "fillColor": { "solid": { "color": "#f9f9f7" } }, "transparency": 0 }]
  }
},
"page": {
  "**": {
    "background": [{ "color": { "solid": { "color": "#f9f9f7" } }, "transparency": 0 }],
    "outspace": [{ "color": { "solid": { "color": "#f0efec" } }, "transparency": 0 }]
  }
}

```

```
}  
}  
}  
}
```

Appendix B — What this build adds to the original guide

Seven things the source build guide does not cover, each found while building against your actual extracts. They are the reason this model gives different answers from a literal reading of the guide.

#	Finding
1	The year-to-date double-count. The February file contains January too. A folder append overstates January by 100% with no error.
2	Two source vocabularies. Volume & Net Spend and Time-in-Transit name the same services differently, and neither list contains the other. Without the bridge, cost and reliability can never meet on one row.
3	The scoring arithmetic. The guide gives four weights across four different units with mixed direction. Added: min-max normalisation across the qualified set, and weight reallocation when a component cannot be measured.
4	Cost must be weight-normalised. Ground averages 16.1 lb per piece against 2.3 lb for 2nd Day Air. Per shipment they differ by 36 cents; per billed pound Ground is 7.3× cheaper.
5	Claims carry no service key, so 50% of the scoring weight cannot be allocated at service grain. The model leaves the join absent rather than approximating it.
6	Statistical confidence, not just a volume floor. Ranking uses the Wilson lower bound, so thin samples fall in proportion to how little is known. Next Day Air Early: 4 packages measured, 100% on-time, confidence floor 51%.
7	Accessorial detail is the only source that explains why cost per piece moves, and it surfaces roughly \$123k a year of avoidable Address Correction charges.